On Theoretical Foundations of Social Laws

Alexander Tuisov

On Theoretical Foundations of Social Laws

Research Thesis

submitted in a partial fulfillment of the

requirements for the degree of Doctor of Philosophy

Alexander Tuisov

Submitted to the Senate of

the Technion - Israel Institute of Technology,

Haifa, Shvat 5783, January 2023

The Research Thesis Accomplished Under The

Supervision of Assoc. Prof. Erez Karpas

at the faculty of Data and Decision Sciences,

Technion

The Generous Financial Help Of The Technion and Daniel Fellowship Is Gratefully Acknowledged

Many thanks are due to Erez Karpas, Alexander Shleyfman, and Michael Katz for their guidance and invaluable help

The author thanks Khorne, Nurgle, Tzeentch, and Slaanesh for guiding him during these chaotic times

The author of this thesis states that the research, including the collection, processing and presentation of data, addressing and comparing to previous research, etc., was done entirely in an honest way, as expected from scientific research that is conducted according to the ethical standards of the academic world. Also, reporting the research and its results in this thesis was done in an honest and complete manner, according to the same standards.

Publications

1. Tuisov, Alexander, and Erez Karpas. ”Automated Verification of Social Law Robustness for Reactive Agents.” ECAI 2020. IOS Press, 2020. 2386-2393.

Contents

Publications

List of Tables

List of Figures

1 Introduction **2**

2 Background **5**

2.1 Preliminaries 5

2.1.1 Multi-Agent Problem Formalism 5

2.1.2 Execution Model 7

2.1.3 Social laws 8

2.1.4 Robustness of a Social Law 10

2.1.5 Labeled Transition System 11

2.1.6 State Space Search 11

2.2 Related Work 12

2.2.1 Social Laws Robustness for Different Formalisms 12

2.2.2 Social law synthesis 13

2.2.3 Multi-agent coordination as FOND 13

2.2.4 Robust normative systems 15

2.2.5 Control via LTL 16

3 Robustness validation **17**
3.1 Previous Compilation 17
3.2 A Counterexample 20
3.3 Corrected Compilation 22
3.4 Formal Proof of Correctness 28

4 Power of Waiting **37**
4.1 Decompositions 37
4.2 The Power of Waiting 45

5 Reactive Agents **49**
5.1 Problem Setup 50
5.1.1 Single Agent Projection 50
5.1.2 Planning and execution model 51
5.1.3 Types of failure 56
5.1.4 Robustness Verification as Reachability Problem 59
5.2 Towards Solving the Reactive Robustness 60
5.3 Hindsight Intent Attribution 61
5.4 Complete Compilation 67
5.5 Empirical Evaluation 70
5.5.1 Summary 72

6 Other Extensions **73**
6.1 Rational Robustness Validation for Templates 73
6.1.1 The Model 74
6.1.2 Compilation 76
6.1.3 Individual Goal Achievement 78
6.1.4 Proof of Correctness 78
6.2 Cost-aware agents 81
6.3 Optimal Robustness 83

7 Conclusion **85**

Bibliography **88**

List of Tables

5.1 Results on BLOCKSWORLD and DRIVERLOG domains with 300 seconds timeout 70
5.2 Experimental setup - grid navigation domain 71
5.3 Results on corridor problems with 300 seconds timeout 71

List of Figures

4.1 Running example: original task with social law that some actions, making the problem decomposable 38
4.2 In this example creating a robust social law is possible only when marking "tile not occupied" preconditions as *waitfor* is enabled. . 45

5.1 Why Internal States Matter. Red arrow is the plan for agent 1, blue arrow is the plan for agent 2. 53
5.2 Graphical representation of the livelock loop from i's point of view. $\pi_i(s_i^0, s)$ is in red, $\pi_i(s_i'', s)$ is in blue, common suffix $\pi_i(s_t) = \pi_i(s_{t-1})$ is in green. Observed actions are in gray. 66

Abstract

Multiple agents operating in a shared environment can interfere with each other's ability to reach their goals. One of the approaches to address this issue is enacting a social law – a set of rules that restricts some possible behaviors of the agents. A social law that ensures that each agent can achieve its goal, regardless of what the other agents do, is called robust.

Recent work has shown how to verify that a given social law, encoded in an MA-STRIPS formalism, is robust by compilation to classical planning. That work also introduced the notion of waitfor preconditions, which assumes that the agent can check if these preconditions hold before executing its scheduled action and withhold from acting otherwise.

In this work, we strive to uncover deeper theoretical foundations of social laws in the context of STRIPS-like environments. We start by pointing out and correcting a mistake in the previous work. Furthermore, we establish the semantics behind the waiting mechanism and connect it to the agent's sensing capabilities. Using these insights, we also derive methods for faster robustness validation. Finally, we reason about the social law robustness in a reactive environment, getting one step closer to real-world scenarios.

Chapter 1

Introduction

Systems with multiple autonomous agents are becoming more and more common (e.g., in robotic fulfillment centers) and will become more so in the future (e.g. autonomous cars and drone delivery). Designing such systems is very challenging; one of the main reasons for this is the need to coordinate all of the agents operating in the same shared environment.

Several approaches for coordination have been explored in the past. One possibility is to use a centralized controller, which controls the actions of all the agents[1]. However, this centralized control is not feasible for a large number of agents, especially when they are owned by different entities (as is the case for autonomous cars). Another approach is to allow each agent to act autonomously and devise "rules of encounter" for when two agents come into conflict, which usually requires some negotiation between the agents. In this work, we follow a

[1] for example, see (Nissim, Apsel, & Brafman, 2012)

third approach, of enacting a "social law" (Shoham & Tennenholtz, 1992) which restricts the behavior of the agents in order to ensure each agent can achieve its own goal. One of the main advantages of social laws is they do not require any communication between the agents.

This work aspires to build a deeper theoretical foundation behind the notion of social laws, with a main focus on MA-STRIPS -like formalisms. We consider a group of self-interested agents who share an environment and may interact with one another, but without any intent of causing harm. Additionally, we assume that there is no communication among the agents. We generally assume that agents have the ability to wait, meaning they can choose to not act while others do. Another assumption is that agents will comply with the social laws the designer enacts. As we are dealing with autonomous and self-interested agents, there is a possibility of non-compliance. To address this, we assume total compliance, which can be interpreted as effective enforcement or the agents' physical inability to engage in non-compliant behavior.

Chapter 2 provides an overview of the necessary background information, including key terms and prior research. In Chapter 3 we point out a mistake in previous research and amend it, resulting in a correct social law robustness validation for a basic MA-STRIPS setting.

In Chapter 4 we dive into expressive power and the theoretical properties of agents' ability to delay the execution of a scheduled action via the *waitfor* mechanism. Our contribution there is twofold: first, we discover that in a setting where no waitfor preconditions are allowed (this may follow, for example, from limited

predicate observability) the robustness property is equivalent to a particular decomposition of a problem. We then explore the environment where some sensing is possible – we first show that there are some tasks that can be made robust *only* by using waitfor preconditions. Second, we prove that adding waitfor preconditions, thus sensing capabilities, never disrupts robustness of a social law.

Chapter 5 is dedicated to the exploration of a setting where replanning is allowed in the execution phase. We propose a suitable definition of robustness for this model and discuss the possible failure modes under this new model. Importantly, when agents are allowed to replan, a new failure mode called *livelock* emerges, in which two or more agents can get stuck in an infinite loop. Moreover, we show how to verify the robustness of a given social law under our new model via a heuristic search on a black-box planning model.

Finally, Chapter 6 contains the social law body of research applied to more exotic settings, such as cost-aware agents and social laws application to templates, as opposed to single problems.

Chapter 2

Background

In this chapter, we provide the required background for the rest of the work. We present the preliminary definitions, as well as the relevant literature review.

2.1 Preliminaries

2.1.1 Multi-Agent Problem Formalism

In this work we primarily deal with problems represented in a modified version of the MA-STRIPS formalism (Brafman & Domshlak, 2008), in which each agent has its own goal (Karpas, Shleyfman, & Tennenholtz, 2017). Such a problem is given by $\Pi = \langle \mathcal{F}, \{\mathcal{A}_i\}_{i=1}^n, I, \{G_i\}_{i=1}^n \rangle$, where $\mathcal{F}$ is the *set of propositions* in the problem Π, $\{\mathcal{A}_i\}_{i=1}^n$ are the *sets of actions* available to *agents* numbered $[n] := \{1, \ldots, n\}$ respectively. Each *action* is a triplet $\langle \mathsf{pre}, \mathsf{add}, \mathsf{del} \rangle$, where all three components are subsets of propositions. Given some action a, we sometimes

refer to $(\mathsf{add}(a), \mathsf{del}(a))$ as $\mathsf{eff}(a)$ for brevity. The *initial state* $I \subseteq \mathcal{F}$ is a subset of propositions, where propositions in it are assumed to be true, and the rest $\mathcal{F} \setminus I$ are false. Finally, each agent $i \in [n]$ has its own *goal* $G_i \subseteq \mathcal{F}$.

The *centralized planning problem* is a STRIPS planning problem (Fikes & Nilsson, 1971) denoted by $\Pi_c = \langle \mathcal{F}, \mathcal{A}_c, I, G_c \rangle$ where $\mathcal{A}_c = \bigcup_{i=1}^{n} \mathcal{A}_i$ and $G_c = \bigcup_{i=1}^{n} G_i$. The *projection* over the set of propositions $X \subseteq \mathcal{F}$ is also a STRIPS planning problem: $\Pi^X = \langle \mathcal{F}^X, \mathcal{A}^X, I^X, G^X \rangle$, where for every proposition subset $E \in \{\mathbb{F}, I, G\}$ we apply the operator $E^X = E \cap X$, e.g., the set of propositions of Π^X is $X = X \cap \mathcal{F}$, and the set of actions is defined as

$$\mathcal{A}^X = \{\langle \mathsf{pre}(a), \mathsf{del}(a) \cap X, \mathsf{add}(a) \cap X \rangle \mid \mathsf{pre}(a) \subseteq X\}.$$

The natural *individual projection* for agent i is denoted by $\Pi_i = \langle \mathcal{F}, \mathcal{A}_i, I, G_i \rangle$.

Let $\Pi = \langle \mathcal{F}, \mathcal{A}, I, G \rangle$ be a STRIPS planning task. We define a state to be $s \subseteq \mathcal{F}$. An action $a \in \mathcal{A}$ can be *applicable* in a state s if $\mathsf{pre}(a) \subseteq s$, and the *result of this application* is $s[\![a]\!] := (s \setminus \mathsf{del}(a)) \cup \mathsf{add}(a)$. The result of sequentially applying (if possible) the sequence of actions π to the state s is denoted by $s[\![\pi]\!]$. The sequence of actions $\pi = a_1..a_m$ is called a *plan* if $G \subseteq I[\![\pi]\!]$. The k-prefix of the plan π is the sequence of actions $a_1..a_k$, and denoted by $\pi|_k$, where $k \in [m]$. The zero prefix applying the empty sequence of actions is annotated as $I[\![\pi|_0]\!] = I$. The set of all plans of a task Π is denoted by $\pi(\Pi)$. A plan π_i for a projection Π_i is called an *individual plan* of an agent i. The plan for Π_c is called a *joint plan*. To ease the notation below we define the following sets for each agent i, let

$\psi : \mathcal{A} \to 2^{\mathcal{F}}$ be one of the following functions $\psi \in \{\mathsf{pre}, \mathsf{add}, \mathsf{del}\}$. We define $\psi(\mathcal{A}_i) = \bigcup_{a \in \mathcal{A}_i} \psi(a)$.

2.1.2 Execution Model

In their work, Karpas et al. (Karpas et al., 2017) introduced the concept of *waitfor*, which constitutes a basic form of synchronization that can eliminate some failures. Informally, *waitfor* is a set of action preconditions that postpones the action execution until all of them are fulfilled at the same time.

Formally, let $\{\pi_i\}_{i=1}^{n}$ be a set of individual agent plans that solve the projection Π_i for each agent $i \in [n]$. The set of *all interleaving executions* is a set of all possible sequences $\pi^{[n]}$ created as follows: for each agent i fix some plan π_i. $\pi^{[n]}$ contains every element $a \in \{\pi_i\}$ exactly once, and each π_i appears in $\pi^{[n]}$ as a subsequence. The set of all interleaving executions is denoted by $\pi^{[n]}(\Pi)$. We say that a planning task is *robust*[1] if $\emptyset \neq \pi^{[n]}(\Pi) \subseteq \pi(\Pi_c)$, i.e. if any interleaving of individual executions is also a joint plan . On one hand, this means that the success of the execution does not depend on the order in which the individual plans are combined, and on the other hand ensures that at least one solution exists. If an action a is to be executed in some state s, but is inapplicable in s, i.e., $\mathsf{pre}(a) \not\subseteq s$, we declare such a situation *failure* and terminate the execution.

Waitfor is a marking of a precondition of an action with the following semantics: during execution, if there is an unfulfilled *waitfor* precondition to the action

[1] This property was called "robust to rational" in the original paper, in this work we use "robust" for clarity

that an agent is about to execute, the agent waits instead. Thus, for each action $a \in \mathcal{A}_c$ the set $\mathsf{pre}(a)$ is divided into $\mathsf{pre}_f(a)$ – regular preconditions, and $\mathsf{pre}_w(a)$ – the $waitfor$ preconditions that signal the agent's executive to postpone the action. Alternatively, $\mathsf{pre}_w(a)$ can be described as creating conditional effects in the following manner: applying action $a = \langle \mathsf{pre}_w, \mathsf{pre}_f, \mathsf{eff} \rangle$ in a state s is equivalent to saying $\mathsf{eff}(a) = \mathsf{eff}$ if $\mathsf{pre}_w \subseteq s$, and $\mathsf{eff}(a) = \emptyset$ otherwise, while still having a as a next action to execute.

Let $\pi_{\text{rw}} = a_1..a_m$ be an interleaving execution of the plans $\{\pi_i\}_{i=1}^n$. We say that π_{rw} respects *waitfor* preconditions if at every execution failure at some $k \in [m]$ it still holds that $\mathsf{pre}_w(a_k) \subseteq I[\![\pi_{\text{rw}}|_{k-1}]\!]$.

We denote the set of all *executions that respect waitfors* of some problem Π as $\pi_{\text{rw}}^{[n]}(\Pi)$.

It is important to note that the actual "decision to wait" is taken only during execution since while planning the agent is not aware of the behaviors of other agents, and thus the global state. In this sense, waitfor preconditions encode a limited fragment of contingent planning (Draper, Hanks, & Weld, 1994), for more on this topic see Brafman and Shani 2012(Brafman & Shani, 2012). The agent itself, however, creates a classical plan and does not need to have contingent planning software onboard.

2.1.3 Social laws

Previous work (Karpas et al., 2017; Tuisov & Karpas, 2020) defined a social law in the context of MA-STRIPS problems in various ways. Here we adopt the most

general definition:

Definition 1 (Social Law) *A social law l is a transformation of an MA-STRIPS task $\Pi = \langle \mathbb{F}, \{\mathcal{A}_i\}_{i=1}^n, I, \{G_i\}_{i=1}^n \rangle$ to a modified task $\Pi^l = \langle \mathbb{F}^l, \{\mathcal{A}_i^l\}_{i=1}^n, I^l, \{G_i^l\}_{i=1}^n \rangle$ such that:*

- $\mathbb{F}^l$ *is a set of propositions.* $\mathbb{F} \subseteq \mathbb{F}^l$
- $\{\mathcal{A}_i^l\}_{i=1}^n$ *is a set of actions for each agent* i. $\forall a^l \in \mathcal{A}_i^l \; \exists! a \in \mathcal{A}_i$:
  - $\mathsf{pre}_f(a) \subseteq (\mathsf{pre}_f(a^l) \cup \mathsf{pre}_w(a^l))$
  - $\mathsf{pre}_w(a) \subseteq \mathsf{pre}_w(a^l)$
  - $\mathsf{add}(a) \subseteq \mathsf{add}(a^l)$
  - $\mathsf{del}(a) \subseteq \mathsf{del}(a^l)$
  - $(\mathsf{add}(a^l) \setminus \mathsf{add}(a)), (\mathsf{del}(a^l) \setminus \mathsf{del}(a)) \subseteq \mathbb{F}^l \setminus \mathbb{F}$
- $I \subseteq I^l \subseteq \mathbb{F}^l$
- $\{G_i^l\}_{i=1}^n \subseteq \mathbb{F}^l$. $\forall i : G_i \subseteq G_i^l$

In plain words, a social law can remove existing actions or add to their preconditions. It may also add propositions to the problem, and effects to the actions, but the added effects can only affect added propositions. This corresponds to the idea that the environment designer cannot add new capabilities to the agents (a city designer cannot make cars fly), but can add auxiliary variables. Lastly, it can add to the goals of agents, for example, to motivate releasing the lock on some resource after using it.

2.1.4 Robustness of a Social Law

A crucial property to look for in social law is its robustness, which is the guarantee that agents operating under the social law will achieve their goals with no conflicts at the end of the execution. We rephrase the definition that appeared in Karpas *et. al.* (Karpas et al., 2017) according to the notation introduced earlier:

Definition 2 (Rational Robustness) *A social law l for multi-agent setting $\Pi = \langle \mathbb{F}, \{\mathcal{A}_i\}_{i=1}^n, I, \{G_i\}_{i=1}^n \rangle$ is rationally robust with respect to waitfors iff: $\emptyset \neq \pi_{rw}^{[n]}(\Pi^l) \subseteq \pi^{[n]}(\Pi_c^l)$.*

In a *rationally robust* task agents can execute their plans in an arbitrary order, and still be guaranteed to arrive at their goals at the end of the execution.

Moreover, *waitfor* preconditions are only accounted for during the execution, not during planning done by agents. Thus, the set of all possible individual plans (and their interleavings) remains the same with no regard to the presence of *waitfor*s. Formally:

Theorem 1 *A rationally robust MA-STRIPS task Π is also rationally robust with respect to waitfors, i.e.:*

$$\emptyset \neq \pi^{[n]}(\Pi) \subseteq \pi(\Pi_c) \implies \emptyset \neq \pi^{[n]}(\Pi) = \pi_{rw}^{[n]}(\Pi).$$

Proof: The direction $\pi_{\text{rw}}^{[n]}(\Pi) \subseteq \pi^{[n]}(\Pi)$ is true by definition. Let $\pi \in \pi^{[n]}(\Pi) \subseteq \pi(\Pi_c)$, then π is a valid plan for Π_c. Let a_k be the k's action in the plan π, then it holds that $\mathsf{pre}(a_k) \subseteq I[\![\pi|_{k-1}]\!]$, which, implies that $\mathsf{pre}_w(a_k) \subseteq I[\![\pi|_{k-1}]\!]$. Thus,

$\pi \in \pi_{\text{rw}}^{[n]}(\Pi)$. ■

2.1.5 Labeled Transition System

In this work, we also make use of labeled (or named) transition systems. As defined by Keller (Keller, 1976), a labeled transition system is a tuple $\langle Q, \rightarrow, \Sigma \rangle$, where Q is a set of states, $\rightarrow$ is a binary relation on Q, also called the set of transitions, and Σ is a set of labels for these transitions. For any $q, q' \in Q$ we denote $q \xrightarrow{\sigma \in \Sigma} q'$ if $(q, q') \in \rightarrow$ is given a label σ. We say that σ is deterministic if there exists only one $q \xrightarrow{\sigma} q'$ for a given q and σ, and the transition system is deterministic if all $\sigma \in \Sigma$ are.

2.1.6 State Space Search

Our tool of choice for solving the task at hand is state space search (Russell & Norvig, 2016). For the purpose of this work, it can be defined as a tuple $\langle S, A, Action(s), Result(s, a), Goal(s) \rangle$ where:

- S is a set of states the system can be in.
- A is a set of action labels.
- $Action(s)$ is a function that establishes the set of actions applicable in some $s \in S$.
- $Result(s, a)$ is a function determining the state reached by applying action $a \in A$ in a state $s \in S$.

- $Goal(s)$ a function determining whether a given state $s \in S$ is a goal of the search.

2.2 Related Work

Several techniques to tackle roughly similar tasks have been made in the past, with varying degrees of success. Thus, here we provide a quick overview of the existing literature in the field.

2.2.1 Social Laws Robustness for Different Formalisms

The concept of social laws has been a subject of investigation in different settings over the years. The earliest exploration of its application took place in the basic setting of the MA-STRIPS task, where recently Karpas et al. (Karpas et al., 2017) proposed an algorithm for robustness validation. The algorithm aimed to compile the robustness validation problem into a classical planning task, but unfortunately contained a mistake which is further elaborated in Chapter 3.

In addition to the basic setting, the notion of social laws was also introduced for continuous-time formalisms (Nir & Karpas, 2019). These adaptations take into account the non-instantaneous duration of actions and their concurrent nature, which are essential considerations in real-world systems. The integration of these aspects bridged the gap between theory and practice, making the concept of social laws more applicable to real-world scenarios.

2.2.2 Social law synthesis

Standalone social law robustness validation is a task that has limited applicability. It can, however be used as a stepping stone towards robust social law synthesis (Nir, Shleyfman, & Karpas, 2020). The synthesis is performed using a heuristic forward search in the space of possible social laws, where the robustness validation procedure serves as a goal test, akin to what was done in the goal recognition design (Keren, Gal, & Karpas, 2019). The authors there propose various ad-hoc heuristics to guide the search toward robust social laws.

This approach, however, suffers from computational issues, as the robustness validation procedure (described in detail in Chapter 3) is as hard as centralized planning, thus undermining the possible computational gains from distributed computation. There were attempts to alleviate this problem by learning a social law from examples of robust laws (Nir, Shleyfman, & Karpas, 2021) with limited success.

2.2.3 Multi-agent coordination as FOND

There are approaches to multi-agent coordination that do not use social laws. For example, a popular approach to solving various multi-agent problems is to examine the system from an agent's perspective. One can model the behavior of other agents as a non-deterministic environment (for example, see (Muise, Felli, Miller, Pearce, & Sonenberg, 2016)), and use fully observable non-deterministic planning (FOND) to find a solution for a particular agent. Concerning the robustness of the

system, one can deduce that if *every* agent has a strong solution, i.e. a solution that succeeds with no regard to outcomes of the non-deterministic action effects, the system is robust.

There are several approaches to single agent non-deterministic planning, and here we will limit our discussion to two of these: *conformant* (Cimatti & Roveri, 2000) and *k-fault tolerant* (Domshlak, 2013) planning.

Conformant planning is an approach to planning under uncertainty that guarantees goal achievement regardless of observations, and by extension, any combination of possible external actions. If every agent in a multi-agent system has a conformant plan, then the eventual arrival of every agent to its goal is guaranteed. On the other hand, k-fault tolerant planning is guaranteed to guide an agent to the goal under the assumption that no more than k faults can occur during the execution. Both k-fault tolerant and conformant planning tasks can be solved using compilation to a heuristic search (Hoffmann & Brafman, 2006).

Both of these methods, however, are efficient only when there are few possible outcomes for each action, while in multi-agent tasks presented as FOND there can be a lot of different outcomes for some actions. Moreover, in k-fault tolerant planning, some outcomes are considered faults, while others are considered primary (or desired). The compilation of k-fault tolerant planning to classical planning assumes each outcome has one primary effect. This is different than our setting, where other agents' actions are their own. Likewise in the multi-agent to FOND compilation, a fixed round-robin execution order between the agents is assumed, which decreases the computational effort significantly, and can not be applied to

reactive robustness verification. Thus, these methods are of very limited use to us.

2.2.4 Robust normative systems

Agotnes *et al.* (Ågotnes, Van Der Hoek, & Wooldridge, 2008) explore the robustness of normative systems. Normative systems are sets of constraints on agents' behavior, and essentially are a special case of social laws. Their notion of robustness is different from what appears in our work: it originates from the assumption that agents can choose not to abide by norms/laws imposed on the system. The authors proceed to explore different aspects of non-compliance and propose ways to deal with it and define *k-robustness* as a property of a normative system that can handle up to k agents that do not comply, and still remain effective. We, however, are not concerned with non-compliance and assume that agents cannot break the social law a designer enacted.

Agotnes and Woolridge (Ågotnes & Wooldridge, 2010) also described a setting where imposing any social law incurs a cost to the system designer, but also provides some benefit. In this context, the authors proceed to describe *optimal social laws* with respect to a given cost function. Thus, designing a social law becomes an optimization problem. In this paper, however, we reason about the properties of an externally given social law and do not attempt to minimize costs of any kind.

2.2.5 Control via LTL

Another way to regulate autonomous agents was done via modeling using Linear Temporal Logic (LTL) controls. These works (Kress-Gazit, Lahijanian, & Raman, 2018; Moarref & Kress-Gazit, 2018) are more robotics-oriented, focusing on navigation, grasping tasks, and swarm behavior. In this line of research, the authors specify agents' behavior via LTL formulae, allowing the agents to be flexible and adaptive in uncertain real-world environments. Due to the computational complexity of LTL, some of its tractable fragments were compiled into formalisms such as GR1 (Bloem, Jobstmann, Piterman, . . . , , & 2012, 2012), of co-safe linear logic by Kupferman and Vardi (Kupferman & Vardi, 2001), producing empirically promising results.

As for the usage of LTL in multi-agent environments, there is a significant body of research on planning and synthesis (Filippidis, Dimarogonas, & Kyriakopoulos, 2012; Guo & Dimarogonas, 2015; Schillinger, Bürger, & Dimarogonas, 2018; Alechina, De Giacomo, Logan, & Perelli, 2022). However, it seems that describing a social law akin to what is presented in this work as an LTL formula is far from trivial, and the size of the resulting expression may be exponential in the size of the MA-STRIPS description. The connection between LTL control and social laws in STRIPS-like formalisms is yet to be fully understood.

Chapter 3

Robustness validation

In this chapter, we uncover and rectify an error in prior research that lies at the core of this thesis. We begin by highlighting the mistake and presenting a clear counterexample that disproves the validity of the robustness validation procedure outlined in (Karpas et al., 2017). We then propose a corrected version and prove its correctness. Finally, we delve deeper into the capabilities of the newly refined compilation method.

3.1 Previous Compilation

Karpas et al. present (Karpas et al., 2017) a way that checks the robustness of a social law within an MA-STRIPS task. They do it by constructing a compilation from a robustness checking problem Π to a classical planning task Π', where Π' has a solution iff Π is not robust. This compilation creates $n + 1$ copies of the

state variables of a given problem Π, n local copies, one per agent, and a global copy. The goal of Π' is formulated such that it matches the failure modes – for example, an action performed in the local copy of some agent that has no matching precondition in the global copy. Each agent then has 3 versions for every action it might take: (1) success, marking the successful application of an action (2) fail, marking the application of an action that has no precondition in the global copy and (3) deadlock, marking an application of an action immediately leading to a state where the agent forever waits for some precondition to become true. We present the full compilation here:

$\Pi' = \langle F', A', I', G' \rangle$, where:

- $F' = \{f_1 \dots f_n \mid f \in F\} \cup \{f_g, f_c \mid f \in F\} \cup \{wt_{f,i} \mid f \in F, i = 1 \dots n\} \cup \{fin_i \mid i = 1 \dots n\} \cup \{failure, act\}$

- $A' = \bigcup_{i=1}^{n} A'_i \cup$
  $\{\text{CHECK-NO-}f, \text{CHECK-NO-WAITING-}f \mid f \in F\}$, where:
  $A'_i = \{\text{END}_i^s, \text{END}_i^f, \text{END}_i^w\} \cup \{a_i^s, a_i^f \mid a_i \in A_i\} \cup \{a_i^{w,x} \mid a_i \in A_i, x \in \mathsf{pre}_w(a_i)\}$, such that:

  $\mathsf{pre}(a_i^s) = act \wedge (\bigwedge_{f \in \mathsf{pre}(a_i)} (f_i \wedge f_g))$,
  $\mathsf{add}(a_i^s) = \{f_i, f_g \mid f \in \mathsf{add}(a_i)\}$,
  $\mathsf{del}(a_i^s) = \{f_i, f_g \mid f \in \mathsf{del}(a_i)\}$,

  $\mathsf{pre}(a_i^f) = act \wedge (\bigwedge_{f \in \mathsf{pre}(a_i)} f_i) \wedge (\bigwedge_{f \in \mathsf{pre}_w(a)} f_g) \wedge (\bigvee_{f \in \mathsf{pre}_f(a_i)} \neg f_g)$,

$\mathsf{add}(a_i^f) = \{failure\} \cup \{f_i \mid f \in \mathsf{add}(a_i)\}$,

$\mathsf{del}(a_i^f) = \{f_i \mid f \in \mathsf{del}(a_i)\}$,

$\mathsf{pre}(a_i^{w,x}) = act \wedge (\bigwedge_{f \in \mathsf{pre}(a_i)} f_i) \wedge \neg x_g$,

$\mathsf{add}(a_i^{w,x}) = \{failure, wt_{x,i}\} \cup \{f_i \mid f \in \mathsf{add}(a_i)\}$,

$\mathsf{del}(a_i^{w,x}) = \{f_i \mid f \in \mathsf{del}(a_i)\}$,

$\mathsf{pre}(END_i^s) = \neg fin_i \wedge (\bigwedge_{f \in G_i} f_i) \wedge (\bigwedge_{f \in G_i} f_g)$,

$\mathsf{add}(END_i^s) = \{fin_i\}$,

$\mathsf{del}(END_i^s) = \{act\}$,

$\mathsf{pre}(END_i^f) = \neg fin_i \wedge (\bigwedge_{f \in G_i} f_i) \wedge (\bigvee_{f \in G_i} \neg f_g)$,

$\mathsf{add}(END_i^f) = \{fin_i, failure\}$,

$\mathsf{del}(END_i^f) = \{act\}$

$\mathsf{pre}(END_i^w) = \neg fin_i \wedge (\bigwedge_{f \in G_i} f_i) \wedge (\bigvee_{f \in F} wt_{f,i})$,

$\mathsf{add}(END_i^w) = \{fin_i, failure\}$,

$\mathsf{del}(END_i^w) = \{act\}$

$\mathsf{pre}(\text{CHECK-NO-}f) = (\bigwedge_{i=1\ldots n} fin_i) \wedge \neg f_g$,

$\mathsf{add}(\text{CHECK-NO-}f) = f_c$,

$\mathsf{del}(\text{CHECK-NO-}f) = \emptyset$

$\mathsf{pre}(\text{CHECK-NO-WAITING-}f) = (\bigwedge_{i=1\ldots n} fin_i) \wedge (\bigwedge_{i=1\ldots n} \neg wt_{f,i})$,

$\mathsf{add}(\text{CHECK-NO-WAITING-}f) = f_c$,

$\mathsf{del}(\text{CHECK-NO-WAITING-}f) = \emptyset$

- $I' = \{act\} \cup \{f_i \mid f \in I, i = 1 \ldots n\} \cup \{f_g \mid f \in I\}$, and
- $G' = \{failure\} \cup \{f_c \mid f \in F\} \cup \{fin_i \mid i \in \{1 \ldots n\}\}$

Unfortunately, the treatment of the $waitfor$ preconditions in this compilation is incorrect and may result in a false negative, i.e., a compilation returns a solution for a robust problem with $waitfor$'s. Closer examination shows that the agents are allowed to affect global copy after performing a deadlock version of an action. We have created a counterexample to the correctness of the compilation, in the form of a robust Π whose compilation is solvable.

3.2 A Counterexample

Here we present the counterexample to the correctness of the above compilation. The counterexample consists of a robust problem Π and a compiled problem Π' that does have a solution. The robust problem Π is as follows:

- P: f^1, f^2, f^3 and their negations.
- A: for agent 1:
  - $a_1^1 : \mathsf{pre}_f = \emptyset, \mathsf{pre}_w = \{f^3\}, \mathsf{add} = \emptyset, \mathsf{del} = \emptyset$

- a_1^2 : $\mathsf{pre} = \emptyset, \mathsf{add} = \{\neg f^1, \neg f^3\}, \mathsf{del} = \{f^1, f^3\}$

for agent 2:

- a_2^1 : $\mathsf{pre} = \emptyset, \mathsf{add} = \{f^2, f^3\}, \mathsf{del} = \{\neg f^2, \neg f^3\}$
- a_2^2 : $\mathsf{pre}_f = \{\neg f^2\}, \mathsf{pre}_w = \emptyset, \mathsf{add} = \{\neg f^3\}, \mathsf{del} = \{f^3\}$

- I: $f^1, \neg f^2, f^3$
- G:
  - $G_1 = \neg f^1$
  - $G_2 = f_2$

The plan in the compiled problem:

1. $a_2^{2,s}$
2. $a_1^{w,3}$
3. $a_2^{1,s}$
4. $a_1^{2,s}$
5. end_1^s
6. end_2^s
7. CHECK-NO-WAITING-$f1$
8. CHECK-NO-WAITING-$f2$

9. CHECK-NO-$f3$

10. CHECK-NO-WAITING-$\neg f1$

11. CHECK-NO-WAITING-$\neg f2$

12. CHECK-NO-WAITING-$\neg f3$

It is easy to validate that the proposed plan solves Π'. To validate that the original Π is robust, it is sufficient to see that no actions have regular preconditions, while the only waitfor precondition of a_1^1 is a goal for agent 2. This means agents cannot interfere with each others' plans. Note that it is also possible to construct an example with no effect-less actions, but it would just complicate the example while adding nothing of substance.

3.3 Corrected Compilation

We amend the compilation by a more careful treatment of the waitfor preconditions, and conflicts in general. We preserve the overall structure of the compilation with n local copies and one global copy of the problem. Our compilation works in two distinct phases: phase 1 is where the planning towards the agents' individual goals, and this phase emulates the execution process. The actions allowed in phase 1 are the agents' actions with three possible outcomes: success, failure, or waiting. The actions of each agent affect both its local and global copy. The main divergence from the compilation above is the introduction of the auxiliary $allow_{(a)}$ flag for each action $a \in A := \bigcup_{i \in [n]} A_i$. This flag is true by default, is

removed by an action application resulting in waiting, and is raised by an action application resulting in success.

Stage 2 is dedicated to the necessary "accounting" making sure that the execution is valid and results in a conflict. In particular, we check whether all agents are either waiting or finished by using a flag fin_i for each agent i. This phase disallows actions that affect the global copy, rather introducing actions that make changes only on the local ones to ensure that each individual plan leads the agent to its goal. The full compilation and the proof of correctness are given below. Let $\Pi' = \langle F', A', I', G' \rangle$ be the compiled problem. We start with defining $F_i = \{f_i \mid f \in F\}$ to be the atoms of each local copy $i \in [n]$, and $F_g = \{f_g \mid f \in F\}$ to be the atoms of the global one. In addition, we define the following set of auxiliary atoms:

$$\begin{aligned} \textit{Aux} = \{&conflict, stage_1, stage_2, \textit{precondition-violation}, \\ &\textit{possible-deadlock}\} \cup \{allow_{(a)} \mid a \in A\} \cup \\ &\{fin_i \mid i \in [n]\}. \end{aligned}$$

Using this notation we define the atoms of Π' to be $F' = Aux \cup F_g \cup \bigcup_{i \in [n]} F_i$. Note that since we use $\neg f_g$ as preconditions for the atoms of the global copy, we assume that both f_g and $\neg f_g$ appear in F' for every $f \in F$. we also keep both $allow_{(a)}$ and $\neg allow_{(a)}$ for each $a \in A$.

We continue with actions of Π'. For each action in A_i we define five distinct

copies,

$$A'_i = \{a_i^s, a_i^f, a_i^w, a_i^l, \text{deadlock-a}_i \mid a_i \in A_i\}.$$

We have three actions that change the local copy of the agent a_i^s, a_i^f, a_i^w. These actions in addition do the following: a_i^s is the successful action that changes the global copy where all agents act in stage 1, a_i^f indicates failure and stops the execution in the global copy ending stage 1 and indicating failure, and the action a_i^l that assures that each agent still achieves its goal in its local copy. The actions a_i^w and deadlock-a$_i$ are actions that are managing waiting, where the first action indicates that agent i is waiting to execute action a_i, and the second indicates that a potential deadlock has occurred.

Additionally, we have a set of auxiliary actions, where end-success$_i$ can be applied if the agent has achieved its goal both in the local and the global copies, an action that allows to execute stage 2, where each agent tries to achieves its goal in the local copy, and three actions that finalize the plan of the compilation with *failure*, the names of these actions are self explanatory.

$$A' = \{\text{start-stage}_2\} \cup \bigcup_{i=1}^{n} (A'_i \cup \{\text{end-success}_i\}) \cup$$
$$\{\text{declare-deadlock}, \text{declare-fail}, \text{goals-not-achieved}\}.$$

Below we present these actions in detail.

- $\mathsf{pre}(a_i^s) = stage_1 \wedge allow_{(a_i)} \wedge \bigwedge_{f \in \mathsf{pre}(a_i)} (f_i \wedge f_g)$,
  $\mathsf{add}(a_i^s) = \{f_i, f_g \mid f \in \mathsf{add}(a_i)\} \cup \{allow_{(a_i)} \mid a_i \in A_i\}$,

$\mathsf{del}(a_i^s) = \{f_i, f_g \mid f \in \mathsf{del}(a_i)\}$;

- $\mathsf{pre}(a_i^f) = stage_1 \wedge allow_{(a_i)} \wedge \bigwedge_{f \in \mathsf{pre}(a_i)} f_i \wedge \bigwedge_{f \in \mathsf{pre}_w(a)} f_g \wedge \bigvee_{f \in \mathsf{pre}_f(a_i)} \neg f_g$,
  $\mathsf{add}(a_i^f) = \{\textit{precondition-violation}, stage_2\} \cup \{f_i \mid f \in \mathsf{add}(a_i)\}$,
  $\mathsf{del}(a_i^f) = \{f_i \mid f \in \mathsf{del}(a_i)\} \cup \{stage_1\}$;

- $\mathsf{pre}(a_i^w) = \{stage_1, allow_{(a_i)}\} \wedge \bigwedge_{f \in \mathsf{pre}(a_i)} f_i \wedge \bigwedge_{f \in \mathsf{pre}_f(a_i)} f_g \wedge \bigvee_{f \in \mathsf{pre}_w(a_i)} \neg f_g$,
  $\mathsf{add}(a_i^w) = \emptyset$,
  $\mathsf{del}(a_i^w) = \{allow_{(a)} \mid a \in A_i \setminus \{a_i\}\}$;

- $\mathsf{pre}(\text{deadlock-a}_i) = allow_{(a_i)} \wedge \bigvee_{f \in \mathsf{pre}_w(a_i)} \neg f_g \wedge \bigwedge_{a \in A_i \setminus \{a_i\}} \neg allow_{(a_j)}$,
  $\mathsf{add}(\text{deadlock-a}_i) = \{fin_i\} \cup \{\textit{possible-deadlock}\}$,
  $\mathsf{del}(\text{deadlock-a}_i) = \{stage_1\}$;

- $\mathsf{pre}(\text{end-success}_i) = \bigwedge_{f \in G_i} f_i$,
  $\mathsf{add}(\text{end-success}_i) = \{fin_i\}$,
  $\mathsf{del}(\text{end-success}_i) = \{stage_1\}$;

- $\mathsf{pre}(\text{start-stage}_2) = \bigwedge_{i \in [n]} fin_i$,
  $\mathsf{add}(\text{start-stage}_2) = \{stage_2\}$,
  $\mathsf{del}(\text{start-stage}_2) = \{stage_1\}$;

- $\mathsf{pre}(a_i^l) = stage_2 \wedge allow_{(a_i)} \wedge \bigwedge_{f \in \mathsf{pre}(a_i)} f_i$,
  $\mathsf{add}(a_i^l) = \{f_i \mid f \in \mathsf{add}(a_i)\} \cup \{allow_{(a)} \mid a \in A_i\}$,
  $\mathsf{del}(a_i^l) = \{f_i \mid f \in \mathsf{del}(a_i)\}$;

- $\mathsf{pre}(\text{goals-not-achieved}) = stage_2 \wedge \bigvee_{i \in [n]} \bigvee_{f \in G_i} \neg f_g \wedge \bigwedge_{i \in [n]} \bigwedge_{f \in G_i} f_i$,
  $\mathsf{add}(\text{goals-not-achieved}) = \{\textit{conflict}\}$,
  $\mathsf{del}(\text{goals-not-achieved}) = \emptyset$;

- $\mathsf{pre}(\text{declare-deadlock}) = stage_2 \wedge \textit{possible-deadlock} \wedge \bigwedge_{i \in [n]} \bigwedge_{f \in G_i} f_i$,
  $\mathsf{add}(\text{declare-deadlock}) = \{\textit{conflict}\}$,
  $\mathsf{del}(\text{declare-deadlock}) = \emptyset$; and

- $\mathsf{pre}(\text{declare-fail}) = stage_2 \wedge \textit{precondition-violation} \wedge \bigwedge_{i \in [n]} \bigwedge_{f \in G_i} f_i$,
  $\mathsf{add}(\text{declare-fail}) = \{\textit{conflict}\}$,
  $\mathsf{del}(\text{declare-fail}) = \emptyset$.

In the initial state we set all the copies to emulate the initial state of Π and declare that the agents currently in stage 1:

$$I' = \{stage_1\} \cup \{f_i \mid f \in I, i \in [n]\} \cup \{f_g \mid f \in I\}.$$

The goal is to achieve *conflict*, i.e., $G' = \{\textit{conflict}\}$.

To remain consistent with the work presented above, we use logic, rather than sets, to express preconditions.

Informally, we emulate the execution process in stage 1, where every agent applies its own actions in an order determined by an external adversarial scheduler. An application of an action during the execution may result in a success, a failure or waiting. In a case of a failure (due to an unfulfilled precondition in the global copy), we proceed to the stage 2 immediately, and only check whether the execution could result from an interleaving of the agents' plans. This is done by checking that every agent can achieve its own goal in the local copy. That way, we assure that an agent could devise a plan from the initial state to the goal state via the state where the conflict is detected. In case agent waits, action a_i^w is applied, and the agent is disallowed to take another action until it performs a_i^w using $allow_{(a_i)}$ predicates.

There are two options for how stage 1 could end – either some agent has an unfulfilled non-waitfor precondition for some action it can apply, which results in raising the *precondition-violation* flag and immediate start of stage 2. The other way is for some agent to declare that it has finished the execution using either deadlock-a$_i$ or end-success$_i$. This way, stage 1 ends, and the bookkeeping begins. Before the start of stage 2, we make sure that every agent has either finished or is waiting for preconditions of some action to become true.

In stage 2, our main purpose is to validate that the execution we emulated in stage 1 could arise from the interleaving of individual agents' plans. This is done by only allowing actions that affect the agent's local copy. If an agent is still able

to reach its goal in the local copy, then there exists an individual plan with a prefix consisting of actions done in stage 1, i.e. the agent was executing a valid plan when the conflict occurred.

To achieve our emulation goal in stage 1, we introduce three types of actions that correspond to the three outcomes of action application: a_i^s for success, a_i^f for failure, and a_i^w for waiting. We also allow the search to continue to stage 2 at any time using the start-stage$_2$ action. In the second stage, we have a_i^l as the local actions, As a deadlock is a state where at least one agent waits at the end of the execution, the preconditions for the declare-deadlock action include the predicate possible-deadlock, which implies the application of at least one of the deadlock-a$_i$ actions. Another way to find a conflict is a non-$waitfor$ precondition violation, which is a prerequisite for declare-fail action. Finally, it may be the case where all agents have finished their respective individual plans with no interruption, but the goal of some of them is not achieved at the end of the execution. We check for this condition using goals-not-achieved action, which performs a straightforward check. The goal predicate *conflict* can be achieved only by these three actions.

3.4 Formal Proof of Correctness

We formally prove the correctness of the new compilation in three main steps. The first step establishes a connection between the executions in the MA-STRIPS task and the plans solving the compiled task. In the second step, we show that if a compiled problem is solvable it means that the underlying problem is not robust.

Finally, we show the opposite direction: if the underlying problem is not robust, then the compiled task is solvable.

Notation: We denote the original MA-STRIPS problem as $\Pi = \langle \mathcal{F}, \{\mathcal{A}_i\}_{i=1}^n, I, \{G_i\}_{i=1}^n \rangle$, and the compiled classical problem as $\Pi' = \langle \mathcal{F}', \mathcal{A}', I', G' \rangle$. For clarity, we add a prime when we refer to a concept from Π' when it is not clear from the context, e.g. π' refers to the plan that solves Π'.

For the sake of the following proofs, we also assume that each agent has at least one plan, i.e., $\pi_{\text{rw}}^{[n]}(\Pi) \neq \emptyset$. Note also that $\pi_{\text{rw}}^{[n]}(\Pi)$ does not contain empty plans, since the first action in each agent's plan is always applicable in the initial state.

Our goal is to provide the connection between the interleavings of agents' individual plans in Π and the trajectories in a compiled problem Π'. We start by establishing the link between trajectories in Π and Π' using the following observation:

Observation 1 *Every plan π' solving Π' must end with either goals-not-achieved, declare-deadlock or declare-fail actions. All of these have $stage_2$ as their preconditions and no action deletes $stage_2$. Moreover, the initial state I includes $stage_1$, no action adds $stage_1$, and every action that adds $stage_2$ deletes $stage_1$ (but not vise versa, some actions delete $stage_1$ without adding $stage_2$). Thus any π' will include a unique action a'_i deleting $stage_1$, a unique a'_j that adds $stage_2$ and it is guaranteed that $i \leq j$. This means, it is always possible to split π' into a prefix $pref(\pi')$, a (possibly empty) infix $inf(\pi')$, and a suffix $suf(\pi')$, where the prefix*

ends with a'_i, and the suffix starts with a'_{j+1}

We now establish a correspondence between an execution of Π and $pref(\pi')$ that tries to emulate it:

Definition 3 (Equivalent sequence and execution) *Let π' be a plan for the compiled problem Π'. Define $\pi^{[n]}$ to be an **equivalent execution of** π' as a projection of $pref(\pi')$ on $F_g = \{f_g \mid f \in F\}$. Note that in this projection, $\mathsf{add}(a_i^w) = \mathsf{del}(a_i^w) = \emptyset$, thus all a_i^w can be safely removed from consideration.*

*Let $\pi^{[n]} = (a_1, \ldots, a_{(m-1)}, a_m)$ be an execution in Π. Define $eq(\pi^{[n]})$ an **equivalent sequence of** $\pi^{[n]}$ in Π' to be $(a_1^s, \ldots, a_{(m-1)}^s)$ if execution fails, otherwise $(a_1^s, \ldots, a_m^s)$.*

F_g is a copy F by construction, thus $\pi^{[n]}$ is consecutively applicable in Π. Moreover, since $pref(\pi')$ includes all actions in π' done before $stage_1$ is removed, $\pi^{[n]}$ is a trajectory in Π. Moreover, $\pi^{[n]}$ is not a prefix of any other interleaving execution. Finally, note that F_g cannot be changed after the removal of the $stage_1$ flag. Since the waitfor actions have been removed, it is also waitfor-respecting. Thus, $\pi^{[n]} \in \pi^{[n]}_{\text{rw}}(\Pi)$.

Similarly, we define local projections of a plan to $F_i = \{f_i \mid f \in F\}$ for each agent i. In a deviation from a global equivalent execution, we use the whole compilation rather than a prefix. The only actions that have an effect on local copy i are a_i^s, a_i^f, and a_i^l, thus every other action can be safely removed from consideration. This corresponds to some individual plan π_i since a plan π' for Π' should achieve local goals for each agent to achieve the $conflict$ atom.

Also, the equivalent sequence of a valid execution is always sequentially applicable in Π', since F_g and every $F_i = \{f_i \mid f \in F\}$ are copies of F, effects of any action a on F are identical to effects of a^s on F_g, and the effects of any a_i are identical to the effects of a_i^s on F_i. Moreover, this means that the state of F and every Π_i after executing some $\pi^{[n]}$ is identical to the state F_g and every F_i respectively after executing $eq(\pi^{[n]})$.

Having established the connection between the executions and the plans in the compiled problem, we are ready to prove the main results using the following theorem:

Theorem 2 (The correctness of the compilation) *Given an MA-STRIPS problem* $\Pi = \langle \mathcal{F}, \{\mathcal{A}_i\}_{i=1}^n, I, \{G_i\}_{i=1}^n \rangle$ *and the complied problem* $\Pi' = \langle \mathcal{F}', \mathcal{A}', I', G' \rangle$ *constructed using compilation shown above,* Π' *is solvable if and only if* Π *is not robust with respect to waitfors, i.e.* $\pi_{rw}^{[n]}(\Pi) \not\subseteq \pi(\Pi_c)$.

Proof: First, we rewrite $\pi_{\text{rw}}^{[n]}(\Pi) \not\subseteq \pi(\Pi_c)$ as $\pi_{\text{rw}}^{[n]}(\Pi) \setminus \pi(\Pi_c) \neq \emptyset$, or in plain words, there exists a $waitfor$-respecting interleaving of individual plans for Π that is not a joint plan.

Lemma 1 Π' *is solvable* $\implies \pi_{rw}^{[n]}(\Pi) \setminus \pi(\Pi_c) \neq \emptyset$.

Proof: Π' is solvable means that there exists a plan π' for Π'. Let β be an execution equivalent to π'. We have to show that such β is not in $\pi(\Pi_c)$, i.e. it is not a joint plan for Π. Moreover, we have to present a set of individual plans π_i that could form β as their interleaving. From Observation 1 we know that π'

includes either goals-not-achieved, declare-fail or declare-deadlock actions. We distinguish between these cases:

Case 1: π' includes goals-not-achieved. By construction, this means that a goals-not-achieved flag was raised. This is only possible if every fin_i was raised, but some goal was not achieved in the global copy. By equivalence of π' and β, at the end of β some global goal will not be achieved as well. Thus, $\beta \in \pi_{\text{rw}}^{[n]}(\Pi)$ but $\beta \notin \pi(\Pi_c)$.

Case 2: π' includes declare-fail. By construction, this means that a precondition-violation flag was raised, thus some action a_i^f was applied, and it was the last action in $pref(\pi')$ since every a_i^f deletes $stage_1$ flag. By equivalence of π' and β, at the last step of the execution action, a_i was not applicable, thus its application would result in a failure. Moreover, every a_i is performed in an individual plan π_i. Such π_i is a local projection of π', where each $a_i \in \pi_i$ has its source in a_i^s, a_i^f or a_i^l, respectively. It is guaranteed that such π_i exists because π' has to achieve $\cup_{i=1}^n G_i$ in the local copies to apply declare-fail.

To sum, $\beta \in \pi_{\text{rw}}^{[n]}(\Pi)$ by equivalence, but $\beta \notin \pi(\Pi_c)$ because applying a_i is both possible and leads to a conflict, and the individual plans that form β are the local projections of π' to F_i for each i.

Case 3: π' includes declare-deadlock. By construction, this means that at least one possible-deadlock flag was raised, thus at least one action deadlock-a$_i$ was applied. This means that action start-stage$_2$ was applied because deadlock-a$_i$ and a_j^f for any agent i, j are mutually exclusive, since both require and remove stage$_1$ flag. The start-stage$_2$ action requires performing either end-success$_i$ or

deadlock-a$_i$ for each i. Since there are no failures, either end-success$_i$ or deadlock-a$_i$ were used. This means, no agent i can apply a successful action because it either has achieved its goal or is in a deadlock. The action end-success$_i$ requires $\bigwedge_{f \in G_i} f_i$, which means in the equivalent β agent i has achieved its goal and stopped the execution. The action deadlock-a$_i$ requires both $\bigwedge_{a \in A_i \setminus \{a_i\}} \neg allow_{(a)}$ and $\wedge \bigvee_{f \in \mathsf{pre}_w(a_i)} \neg f_g$. The first requirement means that the previous action of agent i was a_i^w since otherwise actions in $A_i \setminus \{a_i\}$ would not be disallowed. The second requirement ensures that no action of the type a^s, not even a_i^s can be performed. Hence, during β agent i waits for some precondition to become true. Thus, every agent i is either finished or waiting, meaning that β results in a deadlock.

Similar to Case 2, every a_i is performed in an individual plan π_i. Such π_i is a local projection of π', where each $a_i \in \pi_i$ has its source in a_i^s, a_i^f or a_i^l, respectively. It is guaranteed that such π_i exists because π' has to achieve $\cup_{i=1}^n G_i$ in the local copies to apply declare-fail. Thus, $\beta \in \pi_{\text{rw}}^{[n]}(\Pi)$ by equivalence, but $\beta \notin \pi(\Pi_c)$ since it results in a deadlock. ■

Lemma 2 $\pi_{rw}^{[n]}(\Pi) \setminus \pi(\Pi_c) \neq \emptyset \implies \Pi'$ *is solvable.*

Proof: The solvability of Π' implies that there exists some plan that solves it, i.e. is applicable in the initial state I' and achieves $G' = \{conflict\}$. Denote such plan π'. It is sufficient to specify $pref(\pi'), inf(\pi')$, and $suf(\pi')$. For definitions of $pref, inf, suf$, see Observation 1.

On the other hand, $\pi_{\text{rw}}^{[n]}(\Pi) \setminus \pi(\Pi_c) \neq \emptyset$ implies existence of n individual

plans π_i, one for each agent $i \in [n]$, and their interleaving execution β that ends either without achieving $\cup_{i=1}^{n} G_i$, in a failure, or in a deadlock. For each agent i, either π_i is a subsequence of β, which means i had finished its execution by the end of β, or π_i is not a subsequence of β. Denote the set of all finished agents as FINISHED(β) and define NOT-FINISHED$(\beta) := [n] \setminus$ FINISHED(β). In a case when NOT-FINISHED$(\beta) \neq \emptyset$, we call the suffix of π_i that does not appear in β an unexecuted suffix of i, and we denote by γ some arbitrary interleaving of unfinished suffixes for all $i \in$ NOT-FINISHED(β). Since there is a one-to-one correspondence between the actions of Π and their local copies of actions of Π' we define γ^l to be the local copies of the actions in γ, while preserving their order of application.

Let π_β be some equivalent sequence of β. Recall that π_β is a trajectory in Π'. We aim at reconstructing a plan π' for Π', such that π_β is the prefix of π'. Consider three cases:

Case 1: $\cup_{i=1}^{n} G_i$ is not achieved after executing β and neither failure nor deadlock occurred. Since π_β is the equivalent sequence of β the global copy in the states in Π' correspond to the states of Π, we know that after the execution of π_β, the condition $\bigwedge_{i\in[n]} \bigwedge_{f\in G_i} f_g$ was not achieved, and all agents have finished their executions. Thus, $\bigwedge_{f\in G_i} f_i$ was achieved for each i.

Hence, the full plan π' looks as follows: $pref(\pi') = \pi_\beta$, $inf(\pi') = \left(\|_{i\in[n]}\text{end-success}_i\right) \|$ start-phase-2, $suf(\pi') =$ goals-not-achieved. Here $x \| y$ stands for the concatenation of words x and y. Note that the application of the actions end-success$_i$ can be done in any order, thus any predefined order will do. The plan π' is valid since

it is a trajectory, and the atom *conflict* is achieved.

Case 2: β ends in a failure. Denote the action failing as a^α. The full plan π' will look as follows: $pref(\pi') = \pi_\beta \| a^\alpha$, $inf(\pi') = \emptyset$, $suf(\pi') = \gamma^l \|$declare-fail. This plan is valid because after applying π_β, global preconditions of a^α are not satisfied by the equivalence of π_β. However, local preconditions of a^α are satisfied because otherwise a^α could not be picked by a scheduler to be the next action performed in Π. At the end of π_β the local executions continue. The local actions are executable in arbitrary order since the local copies have no effect on each other (cf. Theorem 2 in the paper). After their execution, it is necessary that $\bigwedge_{f \in G_i} f_i$ is satisfied for each i, otherwise, π_i would not be a plan in Π_i.

Case 3: β ends in a deadlock. This means every agent had either finished its execution or waiting for some $waitfor$ precondition to become true. Then π' will look as follows: $pref(\pi') = \pi_\beta \parallel \left(\|_{i \in \text{NOT-FINISHED}(\beta)} a_i^w \right)$ where a_i is the first action in the unexecuted suffix for each i;

$$\begin{aligned} inf(\pi') = & \left(\|_{i \in \text{NOT-FINISHED}(\beta)} \text{deadlock-a}_i \right) \parallel \\ & \left(\|_{i \in \text{FINISHED}(\beta)} \text{end-success}_i \right) \parallel \text{start-phase-2;} \end{aligned}$$

and $suf(\pi') = \gamma^l \parallel$ declare-deadlock, where all the actions end-success$_i$ and deadlock-a$_i$ can be executed in arbitrary order.

This plan is valid because after executing β, we know that every $i \in$ NOT-FINISHED(β) was waiting for some $waitfor$ precondition to become true. Thus, after applying π_β, for each $i \in$ NOT-FINISHED(β) there will be such an a_i that its regular pre-

conditions are met, but waitfor preconditions are not. Thus, a_i^w are applicable, and can be applied in arbitrary order since they have no effect on each other's preconditions. After that, deadlock-a$_i$ actions become applicable.

As soon as one is applied, $stage_1$ is deleted, and no actions but the deadlock-a$_i$, end-success$_i$, and start-stage$_2$ are applicable.

To achieve $fin_i \forall i$, deadlock-a$_i$ is performed for waiting agents, and end-success$_i$ is performed for finished agents, again in arbitrary order.

After that, start-stage$_2$, and, subsequently local actions become applicable.

The local actions are executable in arbitrary order since they have no effect on each other, and after their execution it is necessary that $\bigwedge_{f \in G_i} f_i$ is satisfied for each i, otherwise, π_i would not be a plan in Π_i. ■

Combining the results of the two lemmas above proves Theorem 1. ■

Equipped with a correct compilation, we proceed to elaborate on the expressive power of the *waitfor* mechanism.

Chapter 4

Power of Waiting

In this chapter, we establish a link between the waitfor mechanism and predicate observability. It is crucial to note that classical and multi-agent classical planning do not consider such mechanisms, and the observability of predicates is a vital aspect of agent and environment design. With a focus on observability issues, we delve deeper into the waitfor mechanism, discovering its connection to contingent planning and evaluating its expressive capabilities.

4.1 Decompositions

To fully grasp the significance and the expressive power of waitfors, it is essential to consider the implications of not using them. In what follows, we will assume the absence of waitfor preconditions. We will then present a practical criterion that demonstrates a problem's robustness.

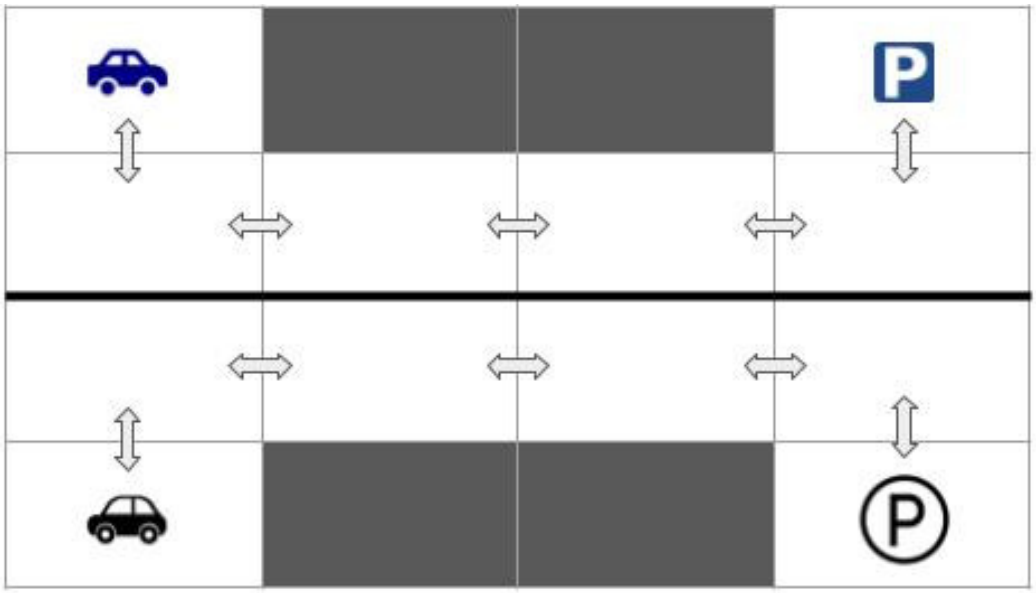


Figure 4.1: Running example: original task with social law that some actions, making the problem decomposable

The main idea behind developing such a criterion is decomposing the multi-agent task into a set of minimally interacting single-agent tasks. If this can be done, and some additional criteria are met – the task is robust.

An informal example is illustrated in Figure 4.1. There, we enact a social law that prohibits any movement of the blue agent below the dividing line, as well as any movement of the black agent above the dividing line. This social law is prohibiting any interference between the agents at all, making the problem robust and decomposable in the sense that we discuss below. Note that a typical multi-agent environment, however, has some agent interaction that cannot be eliminated without hurting the goal achievement of some agents.

To proceed more formally in our effort to characterize robust tasks, we lay out

the relevant definitions:

Given an MA-STRIPS problem $\Pi = \langle \mathbb{F}, \{\mathcal{A}_i\}_{i=1}^n, I, \{G_i\}_{i=1}^n \rangle$, let *decomposition* $\{\mathbb{F}_i\}_{i=0}^n$ be a partition of $\mathbb{F}$ into $n+1$ subsets. The semantics are that $\mathbb{F}_i$ is a set of propositions associated with agent i, and $\mathbb{F}_0$ are the global propositions – for example, the existence of a road usable by all agents.

Given a decomposition, define $D_i = \mathbb{F}_i \cup \mathbb{F}_0$ – the *relevant set* of propositions for agent i. This allows us to define the individual projections for each agent based on their relevant sets as $\Pi_i^{D_i} = \langle \mathbb{F}^{D_i}, \mathcal{A}^{D_i}, I^{D_i}, G^{D_i} \rangle$. We denote by Π^D the set of projections $\{\Pi_i^{D_i}\}_{i=1}^n$. The set of all interleaving plans of Π^D is denoted by $\pi^{[n]}(\Pi^D)$. We say that Π^D is rationally robust if $\pi^{[n]}(\Pi^D) \subseteq \pi(\Pi_c)$. Note that natural projections admit the decomposition $\mathbb{F}_0 = \mathbb{F}$ and $\mathbb{F}_i = \emptyset$ for each $i \in [n]$.

Since $\mathcal{A}_i^{D_i} \subseteq \mathcal{A}_i$ for each $i \in [n]$, a decomposition constitutes a social law in terms of action restrictions. Thus, from all possible decompositions we expose one that has an intrinsic connection to rational robustness:

Definition 4 (Delete decomposition) *Let* $\Pi = \langle \mathbb{F}, \{\mathcal{A}_i\}_{i=1}^n, I, \{G_i\}_{i=1}^n \rangle$ *be an MA-STRIPS problem.* $\{\mathbb{F}_i\}_{i=0}^n$ *is a delete decomposition of* Π *if:*

- $\forall i \in [n] : \mathbb{F}_i = \mathsf{del}(\mathcal{A}_i)$.
- $\mathbb{F}_0 = \mathbb{F} \setminus \bigcup_{i=1}^n \mathsf{del}(\mathcal{A}_i)$.
- $\forall i, j \in [n] \cup \{0\}, i \neq j : \mathbb{F}_i \cap \mathbb{F}_j = \emptyset$

A delete decomposition D_i includes only propositions that no one but the agent i can delete. Such a decomposition is not guaranteed to exist for every problem,

and we label the task *delete-decomposable* if it does have a delete decomposition.

It is worth noting that the concept of delete-decomposability bears some similarity to the definition of private variables used in the context of MA-STRIPS (Brafman, 2015). However, the definition of private variables is much more stringent as it restricts the ability to manipulate these variables to a specific agent, including the add effects. In contrast, our definition does not require the variables to be private, and thus, allows for more flexibility in their manipulation.

We proceed to show that the delete-decomposability of a task, along with some technical requirements, guarantees that the problem in question is rationally robust. To formalize this claim and prove it, we require some additional definitions first:

- **Harmful actions** – actions that do not appear in any individual plan. Note that an unreachable action is always harmful, but not vice versa – for example a reachable action leading to a dead end. An action that is not harmful is called useful.

- **Useless propositions** – propositions that do not appear as preconditions for any useful action.

To illustrate, consider a social law for the example problem (see Figure 4.1) that prohibits only crossing the dividing line, instead of any movement on the wrong side. This makes the blue agent's moving actions on the bottom side of the grid unreachable rather than prohibited, thus rendering them harmful, and also

useless. Although the task is not strictly delete-decomposable as per Definition 4, it is decomposable in practice, while still robust.

Now we have the toolset to formally establish the connection between the delete-decomposability and the rational robustness using the following pair of theorems:

Theorem 3 (Decomposability condition) *Consider an MA-STRIPS problem* $\Pi = \langle \mathbb{F}, \{\mathcal{A}_i\}_{i=1}^n, I, \{G_i\}_{i=1}^n \rangle$. *If* Π*:*

1. *has no waitfor preconditions*
2. *is rationally robust*
3. *has no harmful actions and no useless propositions*
4. *does not contain actions that delete the same proposition they add, i.e.* $\forall a \in \{\mathcal{A}_i\}_{i=1}^n : add(a) \cap \mathsf{del}(a) = \emptyset$

then Π *is delete-decomposable.*

Proof: We need to prove the existence of a delete decomposition $\{\mathbb{F}\}_{i=0}^n$, where each $\mathbb{F}_i = \mathsf{del}(\mathcal{A}_i)$ for each $i > 0$, i.e. it is required to show that $\{\mathbb{F}\}_{i=0}^n$ is a partition. $\mathbb{F}_0$ is always disjoint from the rest of the $\mathbb{F}_i$'s by definition, so it is sufficient to show that $\forall i, j \in [n], i \neq j : \mathbb{F}_i \cap \mathbb{F}_j = \emptyset$

Assume by contradiction that there exists some $f \in \mathbb{F}$ and a pair of agents i, j such that $f \in \mathbb{F}_i \cap \mathbb{F}_j$. Thus, there are two actions $a_i \in \mathcal{A}_i, a_j \in \mathcal{A}_j$ such that $f \in \mathsf{del}(a_i) \cap \mathsf{del}(a_j)$. Given that f is not useless there is some useful action a' such that $f \in \mathsf{pre}(a')$.

Let $a' \in \mathcal{A}_z$. Since there are no harmful actions, a' belongs to some plan of agent z, denoted π_z. Let π_i and π_j be the plans of agents i and j respectively, that include the actions a_i and a_j respectively. Assume, WLOG, that $i \neq z$. There is an interleaving execution where a_i appears immediately before a', resulting in failure in the execution. This contradicts the robustness of Π. ■

The second theorem is an attempt to establish the link in the opposite direction – what is needed for a delete-decomposable task to be rationally robust. Here the conditions are much stricter, since i's goals and the preconditions of i's actions should be in i's control too for the rational robustness to emerge. Formally:

Theorem 4 (Robustness condition) *Consider a delete-decomposable problem with no waitfor preconditions* $\Pi = \langle \mathbb{F}, \{\mathcal{A}_i\}_{i=1}^n, I, \{G_i\}_{i=1}^n \rangle$*, with the relevant sets* $\{D_i\}_{i=1}^n$*. If* $\forall i \in [n] : \mathsf{pre}(\mathcal{A}_i) \cup G_i \subseteq D_i$*, and for each* i *individual task* $\Pi_i = \langle \mathbb{F}, \mathcal{A}_i, I, G_i \rangle$ *is solvable,* Π *is robust.*

Proof: Assume by contradiction Π is not robust. Thus, $\pi^{[n]}(\Pi) \not\subseteq \pi(\Pi_c)$, i.e. execution of some interleaving of individual plans leads to failure or G_c is not achieved.

Goal achievement: for each agent i we know that π_i is a solution, which means if we apply π_i to I, we achieve G_i. Since we assumed $G_i \subseteq D_i$, no element of it can be deleted by action not in $\mathcal{A}_i$. Thus no action from $\mathcal{A}_{j \neq i}$ will have any effect on G_i, which will still be achieved after executing the last action in π_i. This reasoning can be applied to any agent, thus the execution achieves G_c.

Possibility of execution: assume by contradiction that there exists some $\pi =$

$a_1..a_m$ – an interleaving of individual plans that leads to a failure. Let a_k ($k \in [m]$) be the first action that can not be applied. Let $\pi|_{k-1}$ be the $k-1$ prefix of the plan, so $s_{k-1} := s_0[\![\pi|_{k-1}]\!]$. Since a_k is not applicable, it holds that $\mathsf{pre}(a_k) \not\subseteq s_{k-1}$, i.e., there is $f \in \mathsf{pre}(a_k)$ such that $f \notin s_{k-1}$. Such f can be missing from s_{k-1} for three reasons:

1. f was not a part of any state before s_{k-1}, i.e. $f \notin I$, and was never added.

2. f appeared in some state before s_{k-1}, but was last deleted by the same agent i that performs action a_k.

3. f appeared in some state before s_{k-1}, but was last deleted by some agent $j \neq i$.

We now show that all three cases are impossible:

Case 1 and Case 2: Recall that π is an interleaving of individual solutions, i.e. the relative order of actions in π_i is preserved in π. Moreover, we know that π_i is a solution, thus it is executable. This means that after executing the action immediately preceding a_k in π_i, f must be true, hence cases 1 and 2 are impossible.

Case 3: Recall that Π is assumed to be decomposable and $\mathsf{pre}(\mathcal{A}_i) \subseteq D_i$. This means $f \in D_i$. By definition, all propositions that can be deleted by j are in $\mathbb{F}_j$, and also by definition, $\mathbb{F}_j \cap D_i = \emptyset$, i.e., f can not be deleted by any $j \neq i$.

We have shown that execution of any arbitrary π is always possible and always achieves G_c, thus Π is robust. ■

Establishing a connection between robustness and decomposability expands our understanding of systems where sensing is impossible. In those systems, the only way to institute a robust social law is to decompose the multi-agent problem into a multitude of loosely-connected single-agent planning tasks. One can also assume the point of view of a single agent, and model away the behavior of other agents as non-deterministic changes in the environment. Then, a problem is robust iff *every* plan of *every* agent is conformant (Cimatti & Roveri, 2000). It is also worth mentioning that one may relax the notion of robustness to a requirement that *every* agent has *at least one* conformant plan, but this would still require each agent to solve what is essentially a multi-agent planning problem since it needs to find a conformant plan – a plan that works regardless of what the other agents do.

On a side note, the notion of a *waitfor* precondition was introduced in Karpas, Shleyfman, and Tennenholtz (Karpas et al., 2017) as an ad hoc measure to achieve some agent coordination. This measure requires two assumptions that were left implicit:

1. An agent is able to wait, i.e., do nothing.

2. An agent can detect that a precondition was not fulfilled before deciding to wait instead of acting.

To enrich our understanding of the power these assumptions add to the model, let us first consider an environment where the agents cannot sense any preconditions, thus no *waitfor* markings are allowed.

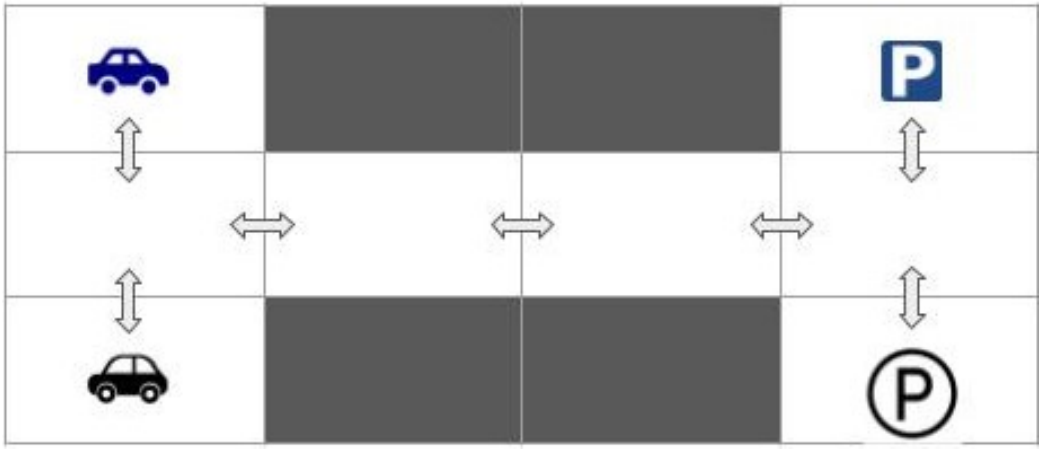


Figure 4.2: In this example creating a robust social law is possible only when marking ”tile not occupied” preconditions as *waitfor* is enabled.

4.2 The Power of Waiting

So far, we limited our discussion to the setting where there are no *waitfor* preconditions for actions. Introducing observability, and subsequently, *waitfor* preconditions, changes the picture. For example, the theorems proved above (3 and 4) are no longer valid. However, in what follows we uncover some deeper connections between robustness and *waitfor*s, as well as their usability and limitations.

Usefulness of waitfors

We slightly modify our running example to convey the usefulness of *waitfors*. As shown in Figure 4.2 using *waitfor* preconditions allows the synthesis of a robust social law. Without *waitfors*, creating a robust social law would be impossible, since both agents need to move to tiles that may be occupied by the other agent.

Consequences of introducing new waitfors

As was shown above, marking preconditions as waitfor can be beneficial, i.e. turn a non-robust problem into a robust one. In what follows we claim that *marking preconditions as waitfors does no harm to robustness*, i.e., it cannot turn a robust problem to a non-robust one. Note that one might think that introducing waitfors has the potential to cause deadlocks. However, we show that in this case, a task would not have been robust in the first place. This claim can be considered a generalization of Thm. 1. Formally:

Theorem 5 *Consider a MA-STRIPS task* $\Pi = \langle F, \{\mathcal{A}_i\}_{i=1}^n, I, \{G_i\}_{i=1}^n \rangle$. *Let* Π^l *be a social law that is identical to* Π *with respect to atoms, initial state, and goals. The actions of* Π^l *are defined as:* $\forall i \in [n], a \in \mathcal{A}_i, \exists! a^l \in \mathcal{A}_i^l$ *such that*

1. $\mathsf{add}(a_i) = \mathsf{add}(a_i^l)$, $\mathsf{del}(a_i) = \mathsf{del}(a_i^l)$;

2. $\mathsf{pre}(a_i) = \mathsf{pre}(a_i^l)$, $\mathsf{pre}_w(a_i) \subseteq \mathsf{pre}_w(a_i^l)$.

Then, $\emptyset \neq \pi_{rw}^{[n]}(\Pi) \subseteq \pi(\Pi_c)$ *implies that* $\emptyset \neq \pi_{rw}^{[n]}(\Pi) = \pi_{rw}^{[n]}(\Pi^l)$.

In plain words, Theorem 5 states that if we have a social law l applied to some task Π such that Π^l only marks some preconditions as *waitfors*, and we know that Π was robust w.r.t. *waitfors*, the sets of all interleavings of all individual plans in Π and Π^l will be identical. The immediate corollary of this theorem is the robustness (w.r.t. *waitfors*) of Π^l.

In what follows, we assume the actions $\mathcal{A}_c$ and $\mathcal{A}_c^l$ to be identical name-wise for the sake of a set of plans comparison.

As the proof goes, we prove equality by proving inclusions in both directions. We first ensure that every interleaving of individual plans in the modified task Π^l is still a valid joint plan for the original task Π. Formally:

Lemma 3 *Let Π and Π^l to be as described in Thm. 5. Then, $\emptyset \neq \pi^{[n]}_{rw}(\Pi^l) \subseteq \pi^{[n]}_{rw}(\Pi)$.*

Proof: Let $\pi^l \in \pi^{[n]}_{\text{rw}}(\Pi^l)$. If π^l is a plan, we have that $\pi \in \pi^{[n]}_{\text{rw}}(\Pi)$, since for each $a^l \in \pi^l$ we have $\mathsf{pre}(a^l) = \mathsf{pre}(a)$. Otherwise, assume in contradiction that $\pi \notin \pi^{[n]}_{\text{rw}}(\Pi)$. Then, there is a failure at the execution of action number k alongside π, denote it a_k. The failure to execute a_k means that there is an atom $f \in \mathsf{pre}_w(a_k)$, such that $f \notin I[\![\pi|_{k-1}]\!]$ but by definition of l we have that $f \in \mathsf{pre}_w(a_k) \subseteq \mathsf{pre}_w(a^l_k)$. This contradicts that $\pi^l \in \pi^{[n]}_{\text{rw}}(\Pi^l)$.

This proves that all interleavings of individual agents' plans in Π^l are also interleavings in Π. To complete the proof, it is sufficient to show the inclusion in the other direction – all interleavings in Π are also interleavings in Π^l.

The direction $\pi^{[n]}_{\text{rw}}(\Pi^l) \subseteq \pi^{[n]}_{\text{rw}}(\Pi)$ is due to Lem. 3. We shall prove inclusion in the other direction. Let $\pi^l \in \pi^{[n]}_{\text{rw}}(\Pi) \subseteq \pi(\Pi_c)$. Since π is a plan for Π_c, there are no conflicts in π. Thus, $\pi^{[n]}_{\text{rw}}(\Pi) \subseteq \pi^{[n]}_{\text{rw}}(\Pi^l)$. ■

Lastly, we establish equivalence between the set of joint plans for Π and Π^l:

Lemma 4 $\pi(\Pi_c) = \pi(\Pi^l_c)$

Proof: Note that Π_c and Π^l_c being classical planning problems are agnostic to waitfors. And without this distinction, $\Pi^l_c = \Pi_c$ by definition. ■

An immediate corollary of Thm. 5 and Lem. 4 is that declaring any precondition of any action as *waitfor* will not invalidate the robustness of the task.

Corollary 1 $\emptyset \neq \pi_{rw}^{[n]}(\Pi) \subseteq \pi(\Pi_c) \implies \emptyset \neq \pi_{rw}^{[n]}(\Pi^l) \subseteq \pi(\Pi_c^l)$.

Note that Corollary 1 implies that introducing a *waitfor* marking cannot create a deadlock (a state where some agents wait for each other indefinitely) in a previously robust system. This means there is nothing to gain from a robustness perspective from leaving some precondition non-*waitfor*.

We have shown that *waitfor* (and therefore sensing), may be used to create a robust social law in systems where it was previously impossible. We have also proven that marking a precondition as *waitfor*, thus requiring an agent to have the ability to sense this precondition preserves robustness. As a consequence, installing additional sensory capabilities to the agents in the system strictly adds to the power of a social law legislator from the robustness perspective.

Limitations of the model

As opposed to mathematical models, in real-world systems, the state of some propositions may not be directly observable by the agents, whereas *waitfor* preconditions require observability. Thus it may be impossible to declare every precondition as *waitfor* in a system. The results presented in this work may help guide the design of intelligent agents with respect to the sensors they need to make certain critical propositions observable.

Chapter 5

Reactive Agents

As was mentioned in the chapters above, previous work has shown how to verify if a given social law is *rationally robust*, that is, ensures that each agent can achieve its goal regardless of the (goal-achieving) plans chosen by the other agents. However, the kind of robustness described in these works is extremely strict — it requires every plan chosen *offline* by every agent to work, regardless of what every other agent is doing. Importantly, this assumes that agents are blind, in the sense that they never change their plan, regardless of what they see the other agents did. This is an extremely strict requirement, and very few real-world problems have meaningful social laws that are *rationally robust*.

In this chapter, we propose a new model, in which agents are *reactive*, that is, they are allowed to *replan* if they notice that their current plan is not going to work. We begin by formulating the execution and replanning model for a reactive environment. We then propose a suitable definition of robustness for this model

and discuss the possible failure modes under this new model. Importantly, when agents are allowed to replan, a new failure mode called *livelock* emerges, in which two or more agents can get stuck in an infinite loop. Finally, we show how to verify the robustness of a given social law under our new model via a heuristic search on a black-box planning model.

5.1 Problem Setup

5.1.1 Single Agent Projection

Since the overall goal of the social law is to decompose the centralized search problem into many much simpler, single-agent search tasks, we have to describe in detail the appropriate single-agent projections. We propose the following single-agent projection of the problem:

Definition 5 ***Reactive single agent projection*** *- given an MA-STRIPS problem* $\Pi = \langle F, \{A_i\}_{i=1}^n, I, \{G_i\}_{i=1}^n \rangle$*, the reactive single agent projection for agent* i *is the planning task* $\Pi'_i = \langle F, A'_i, I, G_i \rangle$*, where* $A'_i = \{\langle (pre(a) \backslash waitfor(a)), add(a), del(a) \rangle \mid a \in A_i\}$

Informally, we ignore the *waitfor* preconditions of the actions to capture the effects of the environment and other agents that are out of i's control. For example, by ignoring the *waitfor* precondition for having some resource for an action, an agent may plan regardless of that resource currently being in use by others. This projection will allow the agent to wait until said resource is available.

In what follows, we say "agent i (re)plans from state s" as a shorthand for "Solving i's reactive single agent projection where the initial state is s". Likewise, we sometimes say "i's plan" as a shorthand for "solution to i's reactive single agent projection".

5.1.2 Planning and execution model

We now describe the planning and execution model for how the agents operate, which allows us to define a new notion of robustness. We distinguish between a number of different settings with regard to replanning while acting.

First, and perhaps the most obvious setting is the one where replanning is forbidden altogether, i.e. every agent plans exactly once before the start of execution. This model is used to derive the notion of *rational robustness* as presented in Definition 2.

Second, we mention the setting where the agent is allowed to replan after every action. Robustness with respect to this model will be denoted as *anytime robustness*. Note that we do not put any limitations on the agent's plan, thus in many domains, the setting contains a trivial livelock akin to the Buridan's ass dilemma.

The next type of model is on a spectrum between the options presented earlier (*never* allow replanning and *always* allow it), as here we allow the agent to replan only *on need*, defined as a state where the agent deduces that the original plan is no longer valid, thus the agent must replan. Replanning *on need* presents a wide range of possible rules for deducing where the need appears, and such an inference

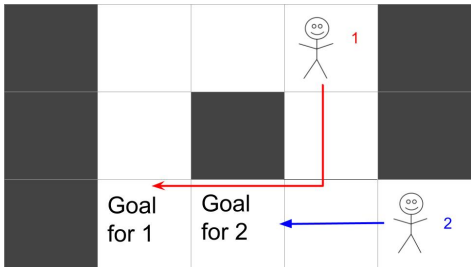
would probably require combining multi-agent reasoning and plan recognition about other agents' actions, which is outside of the scope of this work.

Thus, we would like to focus our discussion here on a specific sub-case of *replanning on need*: *reactive replanning*. In this setting, the agents replan only when they cannot execute the next action in the original plan, i.e. the next action in the plan has an unfulfilled non-*waitfor* precondition when the agent is activated by an external scheduler. This allows us to forgo replanning in situations where the missing precondition of an un-executable action was restored by another agent. Robustness with respect to this model will be denoted as *reactive robustness*.

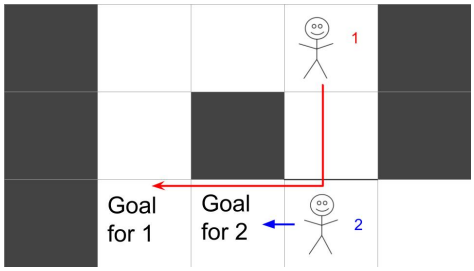
To present a mathematical definition of robustness, we introduce some auxiliary notation and definitions. For a multi-agent setting $\Pi = \langle F, \{A_i\}_{i=1}^n, I, \{G_i\}_{i=1}^n \rangle$, we present a transition system which allows us to reason about agents' plans, which we must do in order to determine when agents are allowed to replan.

This transition system extends the transition system defined by Π by adding an action $replan_i$ for each agent i (in addition to the "regular" actions A_i), which allows the agent to change its current plan. replan$_i$ is only applicable when the preconditions of i's next planned action do not hold. To account for each agent's plan, we also introduce a variable plan$_i$, which holds the current plan of agent i. Thus, the global state of the system is $\langle s, \pi_1, \ldots, \pi_n \rangle$, where $s \subseteq F$ describes the state of the world, and will be called the *world state*. π_i is the not-yet-executed suffix of the current plan agent i intends to execute, and we will refer to it as the *internal state of i*.

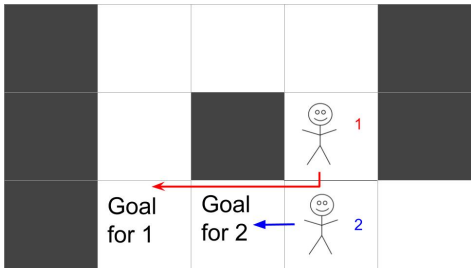
The need to keep track of agents' internal states is best illustrated by an ex-



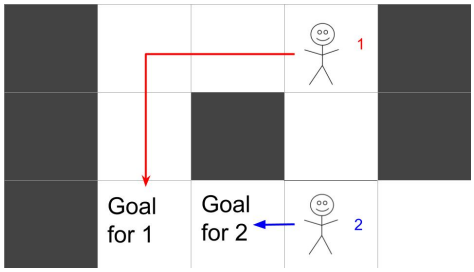
1. Both agents plan.



2. Agent 2 makes a move.



3. Agent 1 comes to a point where it needs to replan.



4. Same world state as (2) with different internal states — no livelock.

Figure 5.1: Why Internal States Matter. Red arrow is the plan for agent 1, blue arrow is the plan for agent 2.

ample as given in Figure 5.1. One of the modes of breaching robustness is the existence of a cycle in the execution system, where agents can repeat the same sequences of actions *ad infinitum*, without ever reaching their goals (see Definition 9 ahead). In this example, although the same world state is reached twice (with different internal states), there exists no cycle agents can be caught in indefinitely. This means the global state g carries additional information, uninferrable from the world state alone.

We now describe the possible transitions in this system from some global state $g = \langle s, \pi_1, \ldots, \pi_n \rangle$. First, any agent i can be chosen to act next, with the exception of agents who are waiting for some *waitfor* precondition, as described below.

Assume agent i's current plan is $\pi_i = \langle a_1^i, a_2^i, \ldots, a_m^i \rangle$. We distinguish between three cases:

1. If a_1^i is applicable in the world state s, that is, pre$(a_1^i) \subseteq s$, then agent i can execute a_1^i, and is not allowed to replan. The system then transitions to the state $\langle (s \setminus del(a_1^i) \cup add(a_1^i)), \pi_1, \ldots \pi_i', \ldots, \pi_n \rangle$. Since the a_1^i was executed, we update i's plan to be $\pi_i' = \langle a_2^i \ldots a_m^i \rangle$.

2. If the non-*waitfor* preconditions of a_1^i hold in s, but some of the *waitfor* preconditions do not, that is, (pre$(a_1^i) \setminus$ waitfor$(a_1^i)) \subseteq s$ and pre$(a_1^i) \not\subseteq s$, then agent i must wait, and another agent will act.

3. If some of the non-*waitfor* preconditions of a_1^i do not hold in s, that is, (pre$(a_1^i) \setminus$ waitfor$(a_1^i)) \not\subseteq s$, then agent i is required to replan instead of performing a_1^i. The system then transitions to the state $\langle s, \pi_1, \ldots, \pi_i', \ldots, \pi_n \rangle$ where π_i' is a possible plan for agent i in the single agent projection from world state s.

There are two facts worth mentioning here: first, an agent is required to apply the first action in its not-yet-executed suffix of a plan. Second, in order for the system to be deterministic, we need to specify the π_i' explicitly every time an agent replans. Formally:

Definition 6 ***Reactive Transition System*** *- given MA-STRIPS task* $\Pi = \langle F, \{A_i\}_{i=1}^n, I, \{G_i\}_{i=1}^n \rangle$ *define the reactive transition system of* Π *as a labeled transition system* $\langle Q, \rightarrow, \Sigma \rangle$ *where:*

- $Q : \{\langle s, \pi_1 \ldots \pi_n \rangle \mid s \subseteq F, \forall i : \pi_i \in \langle a_1^i, \ldots, a_m^i \rangle, a_j^i \in A_i\}$
- $\Sigma = \cup_{i=1}^{n} (A_i \cup \{replan_i(\pi_i') \mid \pi_i' \in \langle a_1^i, \ldots, a_{m'}^i \rangle, a_j^i \in A_i\})$
- $\rightarrow$: $g \xrightarrow{a} g'$ *iff* $g, g' \in Q$, $a \in \{a_1^i \mid i = 1 \ldots n\}$, *and* a *transitions from* g *to* g' *as described in cases 1 - 3 above.*

Note that the number of legal transitions from some states in this transition system is exponential in $|F|$, because of the need to specify π_i'. This presents a computational challenge for reasoning about the reachability in this system. In Section 5.2 we present a way to avoid this problem.

We define a trajectory in the transition system described above as follows:

Definition 7 ***Valid Trajectory*** *- given MA-STRIPS task* $\Pi = \langle F, \{A_i\}_{i=1}^{n}, I, \{G_i\}_{i=1}^{n} \rangle$, *and its reactive transition system* $\langle Q, \Sigma, \rightarrow \rangle$, *a* valid trajectory *is a state – action sequence* $(g^0, a^1, g^1, a^2, \ldots a^n, g^n)$ *such that:*

- $g^0 = (I, \pi_1 \ldots \pi_n)$ *where* π_i *is agent* i*'s plan from* I.
- $\forall i : g^i \xrightarrow{a^{i+1}} g^{i+1}$

Then reactive robustness can be described as follows:

Definition 8 ***Reactive Robustness*** *– A social law* l *for multi-agent setting* $\Pi^l = \langle F^l, \{A_i^l\}_{i=1}^{n}, I^l, \{G_i^l\}_{i=1}^{n} \rangle$ *is reactively robust iff: for all agents* i*, for* ***all*** *valid trajectories* ξ*, execution of* ξ *achieves* $G_1^l \cup \cdots \cup G_n^l$.

Note that the definition requires *any* sequence to lead the agents to their goals, which means any proof of robustness will have to assume an *adversarial* scheduler

deciding which agent will act next, and which plan each agent chooses during replanning.

Let us examine the relation between *rational* and *reactive* robustness. We will establish a hierarchy of robustness types by proving the following theorem:

Theorem 6 *For any problem* Π *and social law* l*, rational robustness is strictly stronger than reactive robustness.*

Proof: Rephrasing the theorem, every Π under l that is rationally robust is also reactively robust, but not vice versa. If Π under l is rationally robust, it is guaranteed that *any* set of the initial plans $\pi_1 \ldots \pi_n$ will be executed to the end without a replanning event. Thus, any valid trajectory ξ will be an arbitrary interleaving of agents' plans, guaranteed to achieve $G_1^l \cup \cdots \cup G_n^l$ by rational robustness.

However, we present an example where reactive robustness $\not\Rightarrow$ rational robustness. Consider the following task: agents $\{Alice, Bob\}$, $F = \{r, g1, g2\}$, $G_{Alice} = g_1, G_{Bob} = g_2$, $I = r$, $A_{Alice} = \{a_1\}, A_{Bob} = \{a_2, a_3\}$, where $a_1 = \langle \emptyset, g_1, r \rangle$, $a_2 = \langle r, g_2, \emptyset \rangle$, and $a_3 = \langle \emptyset, g_2, \emptyset \rangle$. This task is not rationally robust, since if $\pi_{Alice} = (a_1), \pi_{Bob} = (a_2)$ and *Alice* performs a_1 first, Bob's plan will fail. On the other hand, Bob may replan and perform a_3 instead, achieving g_2. Thus the achievement of $\{g_1, g_2\}$ is guaranteed, i.e. this task is reactively robust. ■

5.1.3 Types of failure

Having settled on the execution model, we need to explore in detail what can disrupt robustness. We now discuss the types of failure that could occur during

execution. Reactive robustness can be violated in a few ways:

- Deadend - an agent should act but does not have a defined action to execute for the current state, i.e. an agent cannot execute the next action from its current plan, and the single agent projection is unsolvable. [1]

- Deadlock - A state where no agent can perform any action, i.e. every agent is either waiting or finished and at least one agent is not finished.

- Livelock - a condition where one or more agents change the state of the system continuously, but no agent makes progress towards its goal. The oscillations of the system states continue *ad infinitum*, but the goal of some agents is never achieved.

Note that the types of failure in this work differ from those presented in (Karpas et al., 2017). A *rationally robust* system cannot enter a livelock, since no replanning occurs. On the other hand, in a reactive system an agent cannot fail because of a missing precondition, and will replan instead of declaring failure in that case. Formally, we define them as follows:

Definition 9 *Consider a valid trajectory ξ. We define the above cases as follows:*

Livelock *- ξ leads to a livelock if it is infinite.*

Otherwise, ξ is finite. Denote its last state by g^n.

Deadend *- ξ leads to a deadend if at g^n there exists an agent i such that i's next planned action $\pi_i(g^n)_{[1]}$ is not applicable in the world state $s(g^n)$, i.e.*

[1] It is possible that in the future other agents' actions can make $i's$ plan for the goal possible again, but we still regard a possibility of a deadend as a violation of robustness.

$(pre(\pi_i(g^n)_{[1]}) \setminus waitfor(\pi_i(g^n)_{[1]})) \not\subseteq s(g^n)$*, and i's single agent projection is unsolvable.*

Deadlock *- ξ leads to a deadlock if there are no transitions from g^n and $G_1 \cup \cdots \cup G_n \not\subset s(g^n)$.*

Since we want our robustness verification to be complete, we need to show that the aforementioned failures are the only possible types of failure in our setting. This is done by stating and proving the next theorem:

Theorem 7 *Let l be a social law for multi-agent setting $\Pi^l = \langle F^l, \{A_i^l\}_{i=1}^n, I^l, \{G_i^l\}_{i=1}^n \rangle$. If there does not exist a valid trajectory ξ which leads to a deadend, deadlock, or livelock, then Π^l is reactively robust.*

Proof: Consider a valid trajectory ξ formed by the complete execution of a reactive transition system. If it is infinite - the system is in a livelock by definition. Otherwise, let us examine the final state g^n of ξ. If the system is not in a deadlock, there are two possibilities:

- All the agents have achieved their goals
- There exists at least one agent i that is not waiting or finished, which means it can perform an action. On one hand, if i's next planned action is applicable - i will perform it, contrary to our assumption that g^n was a final state of the system. On the other hand, if i's next action is not applicable - i will replan. If the replanning yielded a new plan - the first action of this plan will be executed, still contrary to our assumption that g^n is a final state. If

the replanning did not yield a plan - the system is in a deadend.

To summarize, g^n can only be the final state of the system, if a system is in a deadlock, deadend, or every agent had achieved its goal. ■

As we have established the failures that lead to a breach in robustness, in the following section we propose a way to check whether a given social law is robust by using a state-space search, where the aforementioned failure modes are the goals of the search.

5.1.4 Robustness Verification as Reachability Problem

Having all the theoretical groundwork covered, we can now describe the reactive robustness verification task as a graph reachability problem. Our goal is to verify there exists a valid trajectory that leads to one of three failure modes. This approach is similar to directed model checking (for example, see (Edelkamp, Lafuente, & Leue, 2001)). Here we only describe the graph, while in the next section, we proceed to describe the full solution. We now explain the vertices, edges, starting vertex, and goal vertices.

Vertices: Vertices in the graph correspond to the global states g the system can be in. The starting vertex corresponds to the starting state g^0 as appears in Definition 7.

Edges: We create a directed edge between two vertices corresponding to g, g' if there exists action a such that $g \xrightarrow{a} g'$ as per Definition 6.

Goals: We search for a counterexample to robustness, thus the reachability

goals correlate with the three modes of failure described earlier. The checks for deadend and deadlock are straightforward and done exactly according to Definition 9. We assume that the plans produced by the agents have no loops, therefore are of bounded length. This restricts our transition system to be finite too. Thus, an infinite execution, and therefore livelock, can form if and only if the graph contains a reachable cycle. This directly correlates to the result shown by Patrizi *et. al.* (Patrizi, Lipoveztky, De Giacomo, & Geffner, 2011) for planning with LTL goals.

Unfortunately, the number of the global states (therefore vertices) is $n^{2^{|F|}}$, and the outdegree of each vertex can reach $2^{|F|}$. The sheer size of the graph precludes us from using straightforward approaches to solve the reachability problem. In the next section, we show how to overcome this obstacle.

5.2 Towards Solving the Reactive Robustness

Our main tool for solving the reachability problem described in Section 5.1.4 is state-space search. However, the state space and the branching factor are prohibitively large, thus a straightforward search in that space will not work.

In order to achieve computational feasibility, we propose a much smaller search space, with a much smaller branching factor while preserving correctness. Specifically, we propose to search in the state space of the *original* MA-STRIPS task (with some minor technical additions), using only actions that correspond to the *original* actions of the agents.

Recall that, as illustrated by Figure 5.1, checking for a livelock requires keeping track of agents' internal states. Additionally, we must make sure that each action an agent executes is one it could have planned to execute when it last replanned. Thus, in order to preserve correctness, we use a sophisticated goal test, which checks whether the sequence of actions which led to the current state is a valid trajectory (Definition 7), and whether one of the failure conditions (Definition 9) occurs.

The complete compilation can be found in Section 5.4. However, we first describe the most complex part of our approach, the planning problem we use to check whether a given trajectory which visits the same world state twice, is a real livelock.

5.3 Hindsight Intent Attribution

To detect a livelock without storing internal states explicitly, we say that a trajectory leads to a *potential livelock* if it encounters the same *world* state for the second time. We then check whether this is a true livelock by finding a plan for each relevant point along the current path where an agent replanned, which justifies the current path and could lead to a livelock.

In more detail, we will require auxiliary definitions of three distinct states for each agent: s_i^0, s_i', s_i'', and one common state: s. These are defined as follows:

- s - a world state where a potential livelock occurred. By definition of the potential livelock it is a world state that occurred more than once during the

execution. Therefore, we can divide the execution to periods according to the visits to s, i.e. refer to the time point of the last visit to the state s as t, and time point of a past visit as $t-1$. We repeat this procedure for each past visit to s, so we allow s to be visited between $t-1$ and t.

- s_i^0 - the last state where agent i planned before $t-1$. Can stem from either i replanning or i planning for the first time in the initial state.

- s_i' - the state where agent i *first* invoked replanning after $t-1$ and before t.

- s_i'' - the state where agent i *last* invoked replanning after $t-1$ and before t. Can be the same as s_i' if i replanned only once between $t-1$ and t.

Since true livelock is equivalent to reaching the same global state for the second time, it is sufficient to show in a potential livelock that the system *could* have been in the same global state, i.e. $\langle s, \pi_1(s_t), \ldots, \pi_n(s_t)\rangle = \langle s, \pi_1(s_{t-1}), \ldots, \pi_n(s_{t-1})\rangle$.

From here on, we look at each individual agent i and reason whether there is *a possibility* that $\pi_i(s_{t-1}) = \pi_i(s_t)$. For that, we divide the agents into three categories with regard to their behavior between s_{t-1} and s_t:

- Agents that did not perform any action between $t-1$ and t : since there was no action performed by those agents, their internal state remains the same. These will be called *irrelevant agents*.

- Agents that performed some actions between $t-1$ and t, but did not replan in this interval : their internal state *must* have changed, which means that

if there is at least one agent in this category, s is not a true livelock. These will be called *advancing agents*.

- Agents that have both performed actions and replanned between $t-1$ and t: for each such agent i we propose to create a planning problem that has a solution if and only if $\pi_i(s_{t-1})$ *could* have been equal to $\pi_i(s_t)$. These will be called *loop agents*.

Any loop agent arrives at s_{t-1} with the plan it conceived at s_i^0. The suffix of this plan is $\pi_i(s_{t-1})$. A second time around, it arrives at s_t with the plan it conceived at s_i'' and has $\pi_i(s_t)$ as a suffix. This means we have to check for the existence of two plans: one from s_i^0 with some prefix up to s (which we will call $\pi_i(s_i^0, s)$) and suffix $\pi_i(s_{t-1})$, and a second plan from s_i'' via s, whose prefix we denote by $\pi_i(s_i'', s)$, such that their suffixes match. Of course, we also require the plans to be consistent with the actions already observed (i.e., executed on the current path). Figure 5.2 provides graphical intuition. We say that the state s had passed the *hindsight intent attribution test* iff such a pair of plans was found for *every* loop agent.

For each loop agent i independently, given its single agent projection Π_i' and the states s, s_i^0, s_i', s_i'' we will construct a classical planning problem Π_i'' that will have a solution iff a pair of plans as described above exists. First, we need both plans to have different prefixes and the same suffix. We split the plans into 4 phases: in phase 1 we follow the observed path ($\pi_i(s_i^0, s)$) from s_i^0 to s, and in phase 2 we follow the observed path ($\pi_i(s_i'', s)$) from s_i'' to s. Then, in phase 3, the

plans merge and reach state s'_i following the observed path $\pi_i(s, s'_i)$, and finally in phase 4, we diverge from the actions that have been observed, and simply need to find a plan that reaches the goal. To keep track of these separate plans, we create 2 copies of the state variables. Actions in phases 1 and 2 affect only one of these copies, while actions in phases 3 and 4 affect both copies. This is similar to the GRD compilation (Keren, Gal, & Karpas, 2015), except with merging instead of splitting. We denote the lengths $|\pi_i(s^0_i, s)|$ by l_1, $|\pi_i(s''_i, s)|$ by l_2, and $|\pi_i(s''_i, s)|$ by l_3, and define the hindsight intent attribution planning compilation as follows:

Given an single agent projection $\Pi = \langle F, A', I, G\rangle$ and states s, s^0_i, s'_i, s''_i, , create a classical planning task $\Pi = \langle F'', A'', I'', G''\rangle$

- F'' = $\{f_1, f_2 \mid f \in F\} \cup \{fin\text{-}ph\text{-}k \mid k \in \{1, 2, 3\}\} \cup \{allow\text{-}a^{ph_k}_j \mid k \in \{1, 2, 3\}, j \in \{1, \ldots, l_k\}$
- A'' = $\{a^{ph_k}_j \mid a \in A, k \in \{1, 2, 3\}, j \in \{1, \ldots, l_k\}\} \cup \{a^{ph_4} \mid a \in A'\}\}$ where:
  - for $k \in \{1, 2\}$:
    $pre(a^{ph_k}_j)$ = $\{f_k \mid f \in \text{pre}(a)\} \cup \{allow\text{-}a^{ph_k}_j\}$,
    $add(a^{ph_k}_j)$ = $\{f_k \mid f \in \text{add}(a)\} \cup \begin{cases} \{allow\text{-}a^{ph_k}_{j+1}\} & j < l_k \\ \{fin\text{-}ph\text{-}k, allow\text{-}a^{ph_{k+1}}_1\} & j = l_k \end{cases}$
    $del(a^{ph_k}_j)$ = $\{f_k \mid f \in \text{del}(a)\} \cup \{allow\text{-}a^{ph_k}_j\}$,
  - $pre(a^{ph_3}_j)$ = $\{f_1, f_2 \mid f \in \text{pre}(a)\} \cup \{allow\text{-}a^{ph_3}_j\}$,

$$add(a_j^{ph_3}) = \{f_1, f_2 \mid f \in \text{add}(a)\} \cup \begin{cases} \{allow\text{-}a_{j+1}^{ph_3}\} & j < l_3 \\ \{fin\text{-}ph\text{-}3\} & j = l_3 \end{cases}$$

$$del(a_j^{ph_3}) = \{f_1, f_2 \mid f \in \text{del}(a)\} \cup \{allow\text{-}a_j^{ph_3}\}$$

– $pre(a^{ph_4}) = \{f_1, f_2 \mid f \in \text{pre}(a)\} \cup \{fin\text{-}ph\text{-}3\}$

$add(a^{ph_4}) = \{f_1, f_2 \mid f \in \text{add}(a)\}$

$del(a^{ph_4}) = \{f_1, f_2 \mid f \in \text{del}(a)\}$

- $I'' = \{f_1 \mid f \in s_i^0\} \cup \{f_2 \mid f \in s_i''\} \cup \{allow\text{-}a_1^{ph_1}\}$
- $G'' = \{f_1, f_2 \mid f \in G\} \cup \{fin\text{-}ph{-}3\}$

To prove the correctness of this compilation, note that the existence of this pair of plans (e.g., a solution to Π_i) indicates that it is possible that $\pi_i(s_{t-1}) = \pi_i(s_t)$ if the agent was acting alone. Moreover, by definition of s_i'' we know that agent i did not replan from s_i'' to s, which means $\pi_i(s_i'', s)$ can be executed regardless of the plans of the other agents. This decoupling gives us the ability to reason about each agent independently, as we can follow the already existing interleaving of the individual plans to achieve the loop we have already observed. Thus, to reach the real livelock it is sufficient to: a) find such a pair of plans for each loop agent and b) show there are no advancing agents.

Theorem 8 *Let s be some state with a potential livelock (that is, search found a path π leading to s, and then looping back to s). Then the hindsight intent attribution for path π is solvable for all agents i iff there really is a true livelock possible from s.*

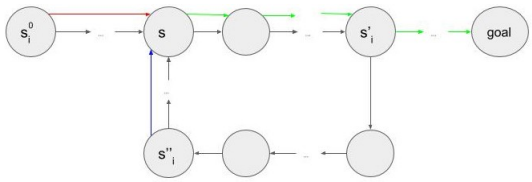


Figure 5.2: Graphical representation of the livelock loop from i's point of view. $\pi_i(s_i^0, s)$ is in red, $\pi_i(s_i'', s)$ is in blue, common suffix $\pi_i(s_t) = \pi_i(s_{t-1})$ is in green. Observed actions are in gray.

Proof: If a true livelock is possible from s, it implies that there exists a schedule of actions such that $\langle s, \pi_1(s_t), \ldots, \pi_n(s_t)\rangle = \langle s, \pi_1(s_{t-1}), \ldots, \pi_n(s_{t-1})\rangle$. This in turn implies that $\forall i : i \notin$ *advancing agents*. Also, for each $i \in$ *loop agents*, there exist two plans: π_i^1 planned before $t-1$ with suffix $\pi_i(s_{t-1})$, and π_i^2 planned after $t-1$ and before t, with suffix $\pi_i(s_t) = \pi_i(s_{t-1})$. Moreover, prefixes of those plans have been already executed, thus are compatible with observed actions. Thus, $\forall i$: π_i^1, π_i^2 is pair of plans that compose a solution for the hindsight intent attribution search procedure in the following way: from the points of the last replans before $t-1$ and t, the actions observed will be executed in their respective copy of facts, until both copies arrive to s. From s, $merge$ will be executed, and after that, merged version of $\pi_i(s_t)$ will complete the solution. We have shown a possible solution for each agent, therefore s passes the hindsight intent attribution test.

Proving the other side of *if and only if*, assume s passed the hindsight intent attribution test. It means, $\forall i \in agents : i \notin$ *advancing agents*. For each $i \in$ *irrelevant agents* trivially $\pi_i(s_t) = \pi_i(s_{t-1})$. For each $i \in$ *loop agents* there exist two plans found by the search procedure: $\pi_i(s_i^0, s) \cdot \pi_i(s_{t-1})$ and $\pi_i(s_i'', s) \cdot \pi_i(s_t)$, where

$\cdot$ denotes concatenation, and $\pi_i(s, s')$ denotes a path from s to s'. Moreover, these plans are consistent with the observed actions, and $\pi_i(s_t) = \pi_i(s_{t-1})$ by the correctness of the search procedure. This, in turn, means that each loop agent *could* have chosen plan $\pi_i(s_i^0, s) \cdot \pi_i(s_{t-1})$ in s_i^0, and plan $\pi_i(s_i'', s) \cdot \pi_i(s_t)$ from s_i'', and still remain consistent with the actions observed, independently of other agents' plans. The independence comes from the fact that both $\pi_i(s_i^0, s)$ and $\pi_i(s_i'', s)$ were executed fully without replanning, i.e. there exists a schedule such that $\forall a \in \pi_i(s_i^0, s), \pi_i(s_i'', s)$, $pre(a)$ are fulfilled regardless of plans of other agents. Thus, there exists a schedule s.t. $(s, \pi_1(s_t), \ldots, \pi_n(s_t)) = (s, \pi_1(s_{t-1}), \ldots, \pi_n(s_{t-1}))$, which means true livelock is possible from s.

■

5.4 Complete Compilation

We present the complete compilation as follows:

Given an MA-STRIPS problem $\Pi = \langle F, \{A_i\}_{i=1}^n, I, \{G_i\}_{i=1}^n \rangle$, define a state space search problem $\Pi^{search} = \langle S, A, Action(s), Result(s, a), Goal(v(s)) \rangle$, where $v(s)$ is a

- S: $s = \langle F, \vec{p}, \{fin_i | i = 1 \ldots n\} \rangle$, where $\vec{p}$ is the vector of replanning events along the path, and fin_i is the flag signaling whether agent i achieved its goal.
- A: $\{activate{-}action{-}a_i \mid a_i \in \{A_i\}_{i=1}^n\}$.

- $Action(s)$: $activate{-}action{-}a_i$: is applicable if $\neg fin_i$.

- $Result(s, a)$ – Given action $activate{-}action{-}a_i$, resulting state s' is constructed as follows:

  – if a_i is applicable in s, i.e. $((pre(a_i) \setminus waitfor(a_i)) \in F(s)) : s' = \langle (F(s) \setminus del(a_i)) \cup add(a_i), \vec{p}(s), fin_i = True$ if $G_i \subseteq (F(s) \setminus del(a_i)) \cup add(a_i), \vec{p}(s), fin_i = False$ otherwise $\rangle$.

  – if a_i is not applicable in s:

    * if i single agent projection is solvable: $s' = \langle F(s), \vec{p}(s) \cdot (i, s), fin_i(s) \rangle$

    * else: $s' = [s, "deadend"]$.

- $Goal(v(s))$: declares $v(s)$ as a goal if $v(s)$ is *justified* and one of the following is true:

  – deadend happens:"$deadend$" in s.

  – deadlock happens: $Action(s) = \emptyset, G_1 \cup \cdots \cup G_n \not\subseteq F(s)$.

  – livelock happens: $s = s'$ for some s' on the path to $v(s)$ and s passes *hindsight intent attribution test*.

In what follows we explain in detail the states, actions, and goals:

States – We keep track of the world states in the set F, auxiliary flags fin_i to denote that i had achieved its goal, and a vector of previously encountered replanning events $\vec{p}$ along the path, which is used in the hindsight intent attribution procedure described earlier.

Actions – we call the actions available to the planner $activate{-}action{-}a_i$, with the semantics of modelling the effect of each a_i available to the agents on the state.

Goals – The goal test examines a search node (as opposed to examining a state), and consists of two parts: in the first we verify that a path to this node is *justifiable*, i.e. the agent *could* have planned to execute the sequence of actions leading it to the node $v(s)$. For that, for agent i that performed a_i leading to $v(s)$ we check if there is a single agent projection solution that starts at the state where i last replanned, and has a prefix consistent with a's observed actions. In the second part of the goal test, we check for the failures from Definition 9. Every action may lead to a deadend, which is checked upon application of each action. The precondition for declaring deadlock follows Definition 9, and the precondition for declaring livelock is arriving to potential livelock and passing the hindsight intent attribution test.

Note that this compilation is much more compact than a reachability problem described in 5.1.4, both in the size of the state space and in the branching factor. The proof of correctness of the complete compilation is quite voluminous, but it closely follows the structure of the compilation itself and is omitted here. The main point of the proof is that every solution of Π^{search} is a valid trajectory and that the goals capture their respective modes of failure correctly.

BLOCKSWORLD			DRIVERLOG		
Problem	Time (sec)	Result	Problem	Time (sec)	Result
9-0	1.49	deadend	1	0.01	robust
9-1	22.75	deadend	2	0.08	deadlock
9-2	12.5	deadend	3	0.06	deadend
10-0	82.07	deadend	4	0.11	robust
10-1	5.28	deadend	5	0.27	deadlock
10-2	170.40	deadend	6	0.43	deadlock
11-0	294.26	deadend	7	1.15	deadlock
11-1	–	timeout	8	0.78	deadlock
11-2	–	timeout	9	0.85	deadend
12-0	–	timeout	10	0.91	robust
12-1	–	timeout	11	0.77	robust
			12	5.16	deadend
			13	47.85	deadlock
			14	16.99	deadlock
			15	81.57	deadend
			16	–	timeout
			17	–	timeout
			18	–	timeout
			19	–	timeout

Table 5.1: Results on BLOCKSWORLD and DRIVERLOG domains with 300 seconds timeout

5.5 Empirical Evaluation

We have implemented the compilation described in Section 5.2, as well as a custom state space planner. The classical planning tasks created for the single agent projection and hindsight intent attribution were solved using *pyperplan*[2].

Following previous work (Karpas et al., 2017), we checked our compilation on the BLOCKSWORLD and DRIVERLOG domains under the random goal distribution procedure, where we created an instance with a different goal set for each agent by allocating each goal to one of the agents at random. Although a direct comparison to the reports provided in (Karpas et al., 2017) is unwarranted, as the rational and reactive robustness are different conditions, we still can draw some parallels since experiments for both were run on the same computers. For the

[2]https://bitbucket.org/malte/pyperplan

agent 1		.....		
		.....		agent 2

Table 5.2: Experimental setup - grid navigation domain

corridor length	5	10	15	20	25	35	45
time (sec)	0.13	3.09	6.46	15.41	28.92	151.44	–

Table 5.3: Results on corridor problems with 300 seconds timeout

BLOCKSWORLD domain, verifying reactive robustness is indeed much slower, yet in DRIVERLOG, verifying reactive robustness is sometimes faster (with the caveat that some problems were robust under the reactive setting, yet not robust for the rational setting). The results appear in Table 5.1.

Since livelocks were not detected in these instances (not because of lack thereof, just because other failures were encountered sooner), we also created a custom set of problems where encountering a livelock is guaranteed, while no other failures exist. The domain is a variant of grid navigation, where an agent can only move on a predefined grid. Movement can be performed only to four adjacent tiles. The agents are forbidden from moving to a tile occupied by another agent, and have a goal of standing on a specific tile.

The set of problems we created features two agents, a corridor of width 2 and of variable length. The agents start on the opposite corners of the corridor, and their goal is to switch places, as illustrated by Figure 3. In Table 5.3 we show the running time of the planner until livelock detection.

Typically, the deadend and deadlock detection occurs much faster than livelock detection, thus for practical use it might be useful to decompose the proce-

dure into two separate steps, where the check for livelock occurs only after no deadlock or deadend has been found.

Note that this evaluation is by no means exhaustive, but it shows that our approach is feasible in practice.

5.5.1 Summary

In this chapter, we have described an execution model which allows agents to adjust their plans *online* via replanning. We then proposed and formally described the notion of reactive robustness of a system under a social law. We also described the problem of checking reactive robustness property as a graph reachability problem. Finally, we have shown how to verify robustness via a compilation to a state space search problem, while keeping the state relatively compact.

Chapter 6

Other Extensions

In this chapter, we explore other extensions to the models proposed before. We focus on two types of extensions: the templates and the cost-aware agents.

6.1 Rational Robustness Validation for Templates

Previous work (Karpas et al., 2017; Nir et al., 2020; Tuisov & Karpas, 2020) has established a framework for verifying the robustness of social laws under various formalisms. Additionally, research has been conducted on the automated synthesis of social laws (Shoham & Tennenholtz, 1992; Nir et al., 2020). However, these studies have only focused on addressing a single problem at a time. This limitation makes it difficult to apply social laws in practice, as small changes in initial conditions or uncertainty about agents' goals can invalidate their robustness.

To address this limitation, we propose a generalized notion of social laws that

takes into account uncertainty in initial conditions and knowledge about agents' goals. We also propose a method for verifying the different types of the robustness of these generalized social laws. By doing so, we aim to reduce the dependency on full knowledge of initial states and goals for each agent and thus make social laws more practical and applicable in real-world scenarios.

6.1.1 The Model

To formalize our attempt in extending robustness validation mechanisms to more than one specific problem, we first provide a formal definition of a generalized problem and a generalized social law. There are multiple ways of compactly specifying a set of MA-STRIPS tasks, the most widely used being an MA-PDDL domain (Kovacs, 2012). Extending social laws to a domain is a complex and challenging process. As a first step, we propose a more limited generalization approach. Specifically, we introduce a template of tasks, which consists of a set of MA-STRIPS tasks with varying initial states and goals. This template ensures that the underlying transition system remains the same for all tasks within a given set. By doing so, we can efficiently apply social laws in slightly different scenarios while minimizing unnecessary complexity.

Definition 10 (MA-STRIPS Template) *MA-STRIPS task template is a 4-tuple* $T = \langle F, \{A_i\}_{i=1}^n, In, \{\Gamma_i\}_{i=1}^n \rangle$*, where* F *and* $\{A_i\}_{i=1}^n$ *defined exactly as in MA-STRIPS.* In *is a set, where each element (possible initial state)* $I_j \in In$ *is a full assignment on* F*. For each agent* i *we define a set of possible goals* Γ_i*, where*

each $G_i \in \Gamma_i$ is a partial assignment on F.

The MA-STRIPS template defines a set of regular MA-STRIPS problems, created from a template by choosing a single initial state I_j from In, and a subset of goals, drawn for each agent i from its set of goals Γ_i. In a slight abuse of notation, we denote a fact that some MA-STRIPS problem Π belongs to a set defined by a certain template T simply as $\Pi \in T$. We proceed to describe the generalized notion of social law.

Definition 11 (Social laws for task templates) *We enact a social law l upon an MA-STRIPS task template T by applying l to each $\Pi \in T$, thus transforming it to Π^l. The set of all such Π^l is denoted T^l.*

We say that the template is *type*-robust if and only if every problem that can be derived from the template is *type*-robust, i.e. T^l is rationally robust iff every $\Pi^l \in T^l$ is rationally robust, and so on. This allows us to formally define the problem we aspire to solve in this work.

Note that any template contains a finite number of tasks, thus verifying the robustness of a template can be naively done by performing an exhaustive check on each task in a template. This, however, is computationally wasteful, thus we propose a more efficient robustness verification mechanism.

As it turns out, the approach to solving template verification depends on the type of robustness in question.

6.1.2 Compilation

We propose to solve rational robustness *for templates* via a compilation to a regular rational robustness verification problem. This means, given an MA-STRIPS *template* $T = \langle F, \{A_i\}_{i=1}^n, In, \{\Gamma_i\}_{i=1}^n \rangle$ we create an MA-STRIPS *problem* $\Pi' = \langle F', \{A'_i\}_{i=1}^n, I', \{G'_i\}_{i=1}^n \rangle$ and check whether it is rationally robust using the compilation proposed in (Karpas et al., 2017). First we explain the intuition behind the compilation, and then we define it formally.

Initial states

We generalize for the multiple possibilities of the initial states by introducing a special flag L and its antipode NL which corresponds to a system starting in a new "limbo" state, where the access to the true initial states is provided by introducing a special actions, one action noted $a_0(I)$ for each $I \in In$. These actions disambiguate the limbo and transition the system into one of the possible initial states. Every action must be performed by some agent, thus we add a dummy agent (numbered 0) for the sole purpose of making a choice of an initial state.

Goal states

Goals are treated differently: for each agent i we create a disjunction between all the elements of Γ_i, which are i's possible goals. Intuitively, if an agent's goal is a disjunction of all its real possible goals, it can choose which goal to pursuit. Since

the verification mechanism is adversarial, the verifier is able to choose specific goal combination that leads to a conflict if one exits. The goal of the dummy agent is NL, since its only purpose is to disambiguate an initial state.

Formally:

Definition 12 (Compilation for rational robustness) *Given an MA-STRIPS template modified by social law* $T^l = \langle F, \{A_i\}_{i=1}^n, In, \{\Gamma_i\}_{i=1}^n \rangle$ *we create an MA-STRIPS problem* $\Pi' = \langle F', \{A'_j\}_{j=0}^n, I', \{G'_j\}_{j=0}^n \rangle$. *The components of* Π' *are defined as follows:*

- F' – $F \cup \{L, NL\}$
- A'_j – *For* $j \in [n]$:

  $\forall a \in A_j \; \exists! \alpha \in A'_j$:

  - $pre(\alpha) = pre(a) \cup \{NL\}$
  - $waitfor(\alpha) = waitfor(a)$
  - $add(\alpha) = add(a)$
  - $del(\alpha) = del(a)$

  For $j = 0$:

  $\forall I \in In \; \exists! a_0(I) \in A'_0$:

  - $pre(a_0(I)) = \{L\}$
  - $waitfor(a_0(I)) = \emptyset$

- $add(a_0(I)) = I \cup \{NL\}$
- $del(a_0(I)) = \{L\}$

- $I' - \{L\}$
- $G'_i - \bigvee_{G \in \Gamma_i} G$ *for* $i \in \{1 \ldots n\}$. $G'_0 = \{NL\}$

We refer to Π' as a problem *derived* from T. We also refer to actions $a \in A_i$ and $\alpha \in A'_i$ as described above as *equivalent actions*. Note that by construction for any action in T there exists exactly one equivalent action Π' and vice versa.

6.1.3 Individual Goal Achievement

The definition of a rationally robust social law (Def. 2) requires any agent to be able to reach a goal state if it acts alone in the environment. We propose a straightforward check of this property: for each agent i, for each initial state $I \in In$ and for every possible goal in $G \in \Gamma_i$, we check whether G is reachable from I using only actions from A_i. Total amount of such reachability checks needed is at most $n \times |I| \times \sum_{i=1}^{n} |\Gamma_i|$. Note that these checks involve solving a single-agent task, thus are assumed to be easy. If at least one of the reachability analyses fails, we say that the procedure failed and the template in question is not robust.

6.1.4 Proof of Correctness

First we formally state that the compilation presented above is correct:

Theorem 9 *An MA-STRIPS template T is rationally robust if and only if the MA-STRIPS problem Π' derived from it via the compilation described in definition 12 is rationally robust.*

Proof: Since the claim is *if and only if*, we show that both directions hold. We first need to provide some more auxiliary definitions:

Definition 13 (Equivalent sequence of actions) *Consider some template $T = \langle F, \{A_i\}_{i=1}^n, In, \{\Gamma_i\}_{i=1}^n \rangle$, some task $\Pi \in T$, and problem $\Pi' = \langle F', \{A'_j\}_{j=0}^n, I', \{G'_j\}_{j=0}^n \rangle$ derived from T. For each sequence of actions $\pi = (a^1, a^2, \ldots, a^m)$ for Π , define an equivalent sequence of actions π' for Π' as $(\alpha^0, \alpha^1, \alpha^2, \ldots, \alpha^m)$, where $add(\alpha^0) = I(\Pi) \cup \{NL\}$, and for $i \in \{1, \ldots, m\}$, α^i is an equivalent action to a^i.*

Note that for each plan there exists exactly one equivalent plan, since α^0 is uniquely defined by its add effect, and for each a exists exactly one equivalent α. We can now define an equivalence function for a sequence of actions:

Definition 14 (Equivalent sequence function) *Given a task $\Pi \in T$ and Π' a derived problem for T, let $EQ : (a_1, \ldots a_m) \mapsto (\alpha_0, \ldots \alpha_m)$ be an equivalent sequence function, where each a is an action in Π, each α is an action in Π', and $(a_1, \ldots a_m)$ is equivalent to $(\alpha_0, \ldots \alpha_m)$.*

Note that EQ is invertable, since for each action there exists only one equivalent action, and we can reconstruct $I(\Pi)$ from $add(\alpha_0)$. We denote the inverse of $EQ(\cdot)$ as $EQ^{-1}(\cdot)$.

Having all the necessary tools, we now assert that for every plan π solving some $\Pi \in T$ the sequence $EQ(\pi)$ applied to Π' is also a plan, and if some π' is a plan for Π', $EQ^{-1}(\pi')$ is a plan for some $\Pi \in T$. Formally we present the following two lemmas:

Lemma 5 *Let π be a joint plan for some $\Pi \in T$ and $\pi' = EQ(\pi)$. π' is a plan for Π', i.e. $\bigcup_{j=0}^{n} G'_j \subseteq I'[\![\pi']\!]$.*

Proof: By Def. 12 G'_i – $\bigvee_{G \in \Gamma_i} G$ for $i \in \{1 \ldots n\}$, while $G'_0 = \{NL\}$. Consider α^0, the first action in π'. Since $add(\alpha^0) = I(\Pi) \cup \{NL\}$ by definition, G'_0 is achieved. Moreover, since $I' = \{L\} = del(\alpha^0)$, $I'[\![\alpha^0]\!] = I(\Pi) \cup \{NL\}$. Recall that for $i > 0$, $pre(a^i), add(a^i), del(a^i) = pre(\alpha^i), add(\alpha^i), del(\alpha^i)$ respectively, thus $s[\![a^i]\!] = s[\![\alpha^i]\!]$. By induction, we get that $I'[\![\pi']\!] = I[\![\pi]\!] \cup \{NL\}$. Since π is a plan, $G^n_{j=1}(\Pi) \subseteq I[\![\pi]\!] \subseteq I'[\![\pi']\!]$. Since $G^n_{j=1}(\Pi) \in \bigvee_{G \in \Gamma_i} G$, G'_i is achieved. ■

Lemma 6 *Let π' be a joint plan that solves Π' and $\pi = EQ^{-1}(\pi')$. $\exists \Pi \in T : \pi$ solves Π, i.e. $\bigcup_{i=1}^{n} G_i(\Pi) \subseteq I(\Pi)[\![\pi]\!]$*

Proof: Let α^0 be the first action of π'. $add(\alpha^0)$ is of a form $I \cup \{NL\}$ by definition, where I is a possible initial state for some $\Pi \in T$. As was shown in the proof of Lem 5, if π and π' are equivalent, then $I'[\![\pi']\!] = I[\![\pi]\!] \cup \{NL\}$. Moreover, $\bigvee_{G \in \Gamma_i}$ is achieved by execution of π', which by definition means that $\forall i \in \{1 \ldots n\} \exists G_i \in \Gamma_i$ such that $G_i \subseteq I'[\![\pi']\!]$. Then $\{G^n_{i=1}\} \subseteq I[\![\pi]\!]$, thus $\Pi = \langle F, \{A_i\}^n_{i=1}, I, \{G^n_{i=1}\}\rangle \in T$ is solved by π. ■

To summarize: Lem. 5 asserts that if π is a plan for $\Pi \in T$, $EQ(\pi)$ is a plan

for Π', and Lem. 6 asserts that if π' is a plan for Π', $EQ^{-1}(\pi')$ is a plan for some $\Pi \in T$. Analogous arguments can be made for individual plans instead of joint plans. We omit the proofs here for brevity, but they follow exactly the same logic.

Now we finalize the proof of Thm. 9:

■

6.2 Cost-aware agents

The previous chapters have primarily focused on synthesizing robust social laws within STRIPS-like formalisms, while assuming that agents are indifferent to the costs of their plans, which can lead to interference between agents. Here we adopt a more realistic assumption that agents take into consideration the cost of their plans. This presents an interesting trade-off: while agents will be less inclined to interfere with each other, achieving robustness becomes more difficult as they may not accept a law that significantly increases their costs.

We introduce a new concept of robustness that is suited for cost-sensitive agents and demonstrate the robustness verification procedure. We also investigate the impact of this trade-off on the synthesis of social laws.

Here we consider a fragment of numeric planning called *restricted numeric tasks* (Hoffmann, 2003). Formally, a restricted numeric task is defined as a 5-tuple: $\langle \mathcal{F}_p, \mathcal{N}, A, s_I, G \rangle$, where:

- $\mathcal{F}_p$ is a set of Boolean propositions.
- $\mathcal{N}$ is a set of numeric variables with rational values.

We define the states of this formalism to be $s = s_p \cup s_n$. Here $s_p \subseteq \mathcal{F}_p$ is a subset of Boolean propositions, and s_n is a full assignment on variables in $\mathcal{N}$. The value of a variable v in the state s is written as $s[v]$. We also define conditions on state s. If s satisfies the condition ψ, we write $s \models \psi$. A Boolean condition ψ is simply a subset of $\mathcal{F}_p$, and $s \models \psi$ iff $\psi \subseteq s_p$. In turn, a numeric condition is given by a formula $v \leq w$, where $v \in \mathcal{N}$ and $w_v \in \mathbb{Q}$. s satisfies $v \leq w$ iff $s[v] \leq w$.[1]

- A is a set of actions, where each action $a \in A$ is a triplet $\mathsf{pre}(a), \mathsf{eff}(a), \mathsf{cost}(a)$. $\mathsf{pre}(a)$ are the preconditions of action, which are subdivided into proposition and numeric preconditions. We write this as $\mathsf{pre}(a) = \mathsf{pre}_p(a) \cup \mathsf{pre}_n(a)$, where $\mathsf{pre}_p(a) \subseteq \mathcal{F}_p$ and $\mathsf{pre}_n(a)$ is a set of numeric conditions. a is applicable in s if $s \models \mathsf{pre}(a)$. $\mathsf{eff}(a)$ are the effects of the of an action a, and are represented by a 3-tuple $\mathsf{eff}(a) = \langle \mathsf{add}(a), \mathsf{del}(a), \mathsf{num}(a) \rangle$. $\mathsf{add}(a) \subseteq \mathcal{F}_p$ and $\mathsf{del}(a) \subseteq \mathcal{F}_p$ are the sets of propositions that become true and false respectively as a consequence of a. $\mathsf{num}(a)$ are the numeric effects of the form $v \mathrel{+}= k$, where $v \in \mathcal{N}$ and k is some constant in $\mathbb{Q} \setminus \{0\}$. The result of the application of a in s is denoted by $a[\![s]\!]$. In this case, $a[\![s]\!]_p = (s \setminus \mathsf{del}(a)) \cup \mathsf{add}(a)$ for the propositional part, $a[\![s]\!]_n[v] = s[v] + k$ if $v \mathrel{+}= k \in \mathsf{num}(a)$ and $a[\![s]\!]_n[v] = s[v]$ otherwise.

- s_I is the initial state.

[1] In the original formalism, the authors use the $\geq$ conditions and show that other conditions can be brought to this form. We, however, use the less-or-equal conditions since they are more fitting to our setting. As we show later on, all numeric variables in our setting are bounded in some interval $[0, w_{\max}]$. In this sense, it also can be seen as STRIPS with resource variables proposed by Nakhost et al. (Nakhost, Hoffmann, & Müller, 2012) s satisfies a set of conditions Ψ if $s \models \psi$ for each $\psi \in \Psi$.

- $G = G_p \cup G_n$ is a set of propositional and numeric conditions, respectively. s is a goal state if $s \models G$.

6.3 Optimal Robustness

Previous work on social laws disregards agents' costs entirely and focuses solely on ensuring robustness. We consider such a focus an unrealistic relaxation. In many setting agents care about their costs, and will not go out of their way just to preclude other agents from performing their actions. With the prevalence of efficient optimal planners nowadays[2], it is not unreasonable to assume any cost-conscient agent would use one. This has direct implications for the robustness of the social laws, since restricting agents to optimal plans will reduce the number of plans and their possible interleavings, making robustness, a bit counterintuitively, *easier* to achieve.

Formally, we consider a setting where each action a_i has some non-negative cost $c(a_i)$ associated with it. We assume additive costs, i.e. agent's total cost is equal to the sum of all actions in its plan. We denote a cost of an individual plan π_i as $cost(\pi_i)$ and we define an optimal plan as $\pi_i^* = argmin_{\pi \in \{\pi_i\}}(cost(\pi))$. In line with our previously introduced notation, we denote as $\pi^*_{[n]}(\Pi)$ the set of all possible interleavings of *optimal* plans. Please note that the set of all optimal plans is a subset of a set of all plans, thus $\pi^*_{[n]}(\Pi) \subseteq \pi^{[n]}(\Pi)$ for any task Π. Now we can formally define *optimal robustness* analogously to the definition of rational

[2]for example Delfi (Katz, Sohrabi, Samulowitz, & Sievers., 2018)

robustness. The definition is as follows:

Definition 15 (Optimal Robustness) *An MA-STRIPS task is optimally robust iff* $\emptyset \neq \pi^*_{[n]}(\Pi) \subseteq \pi(\Pi_c)$

This notion of robustness is strictly laxer than rational robustness, since on one hand, we prohibit non-optimal behavior, decreasing the size of the set of possible plans, and therefore decreasing the number of possible conflicts. On the other hand, if an agent has *a* plan, it also has an optimal plan, thus we cannot render the individual projections unsolvable for some agents by requiring optimality. Formally we state the following:

Theorem 10 *If MA-STRIPS task* Π *is rationally robust, it is also optimally robust, i.e.* $\emptyset \neq \pi^{[n]}(\Pi) \subseteq \pi(\Pi_c) \rightarrow \emptyset \neq \pi^*_{[n]}(\Pi) \subseteq \pi(\Pi_c)$

Proof: On one hand, $\forall i$: if $\{\pi_i\} \neq \emptyset$ then $\pi^*_i \in \{\pi_i\}$, thus $\pi^*_{[n]}(\Pi) \subseteq \pi^{[n]}(\Pi)$. On the other hand, the requirement $\emptyset \neq \pi^{[n]}(\Pi)$ assures no $\{\pi_i\}$ is empty, and since $\{\pi_i\}$ is finite, it has a minimal element with respect to cost, i.e. if $\pi^{[n]}(\Pi) \neq \emptyset$ then $\pi^*_{[n]}(\Pi) \neq \emptyset$.

Thus, $(\emptyset \neq \pi^{[n]}(\Pi) \subseteq \pi(\Pi_c)) \rightarrow (\emptyset \neq \pi^*_{[n]}(\Pi) \subseteq \pi^{[n]}(\Pi) \subseteq \pi(\Pi_c))$. ■

Chapter 7

Conclusion

In sum, this thesis aims to establish a more robust theoretical foundation for social laws in multi-agent deterministic formalisms. The focus is on a group of self-interested agents who share an environment and interact with one another.

In Chapter 3 we refined the previous work on the social law robustness validation by amending the compilation and formally proving the correctness of our version.

In Chapter 4 we establish the connection between sensing and the *waitfor* preconditions within the framework of social law design. We show that if the system does not support observing predicates, and thus, cannot facilitate *waitfor* preconditions, its capacity for implicit coordination via robust social laws is limited to *delete-decomposability*, i.e., the multi-agent planning task can have a social law if and only if each agent can be limited to its planning task that is not related to other agents. Moreover, we demonstrate that the presence of observable predicates is

strictly beneficial since it cannot harm the robustness of the law. On one hand, this leads to the conclusion that the main limitation of social law design is access to sensors and observation, which enables implementation of *waitfor* preconditions of the agent. On the other hand, this may guide the design of agents' sensor installation to promote the creation of a robust social law.

The overarching goal of research on social laws is to create robust social laws for different environments. This work has direct implications for the problem of social law synthesis. In an environment where sensing is impossible, we provide a new way of checking for rational robustness. To conclude that an MA-STRIPS problem with no $waitfors$ is robust all we need is to check for conditions of Theorem 4. This problem boils down to checking two non-trivial conditions: (1) any agent has an individual plan, and (2) there are no harmful actions and useless preconditions. While checking individual solvability still requires solving multiple instances of single-agent planning problems, which is PSPACE-complete, it should be much easier in practice than solving the centralized planning problem, as was required in previous work (Brafman & Domshlak, 2008). This is because the plan for the centralized problem contains the individual plans of all agents as subsequences. It is sufficiently easy to see that checking for harmful actions is also PSPACE-complete. However, since it is done in the individual projection of each agent, the overall trade-off between our method and the one proposed by Karpas et al. becomes clearer. While Karpas et al. solve a classical planning problem that consists of $n + 1$ copies of the original MA-STRIPS problem, we propose to solve $|\mathcal{A}| + n$ smaller problems, where each problem is solved on the individual

projection of some agent. We hypothesize that this method should perform better in practice since it takes advantage of implicit partial order reduction, however, if this is the case remains to be seen.

We also have described an execution model which allows agents to adjust their plans *online* in Chapter 5. We then proposed and formally described the notion of reactive robustness of a system under a social law. We also described the problem of checking reactive robustness property as a graph reachability problem. Finally, we have shown how to verify robustness via a compilation to a state space search problem, while keeping the state relatively compact. Finally, we have presented a proof-of-concept empirical evaluation, and have shown that our procedure works on some benchmark problems.

Bibliography

Ågotnes, T., Van Der Hoek, W., & Wooldridge, M. (2008). Robust normative systems. In *Proceedings of the International Joint Conference on Autonomous Agents and Multiagent Systems, AAMAS*, Vol. 2, pp. 733–740.

Ågotnes, T., & Wooldridge, M. (2010). Optimal social laws. In *Proceedings of the International Joint Conference on Autonomous Agents and Multiagent Systems, AAMAS*, Vol. 1, pp. 667–674.

Alechina, N., De Giacomo, G., Logan, B., & Perelli, G. (2022). Automatic Synthesis of Dynamic Norms for Multi-Agent Systems. In *19th International Conference on Principles of Knowledge Representation and Reasoning*.

Bloem, R., Jobstmann, B., Piterman, N., . . . , A. P. J. o. C., , & 2012, U. (2012). Synthesis of reactive (1) designs. *Journal of Computer and System Sciences*, *1*.

Brafman, R. I. (2015). A privacy preserving algorithm for multi-agent planning and search. In *IJCAI International Joint Conference on Artificial Intelligence*, Vol. 2015-January.

Brafman, R. I., & Domshlak, C. (2008). From one to many: Planning for loosely coupled multi-agent systems. In *ICAPS 2008*, pp. 28–35.

Brafman, R. I., & Shani, G. (2012). A multi-path compilation approach to contingent planning. In *Proceedings of the National Conference on Artificial Intelligence*, Vol. 3, pp. 1868–1874.

Cimatti, A., & Roveri, M. (2000). Conformant planning via model checking. *Journal of Artificial Intelligence Research*, *1809*, 21–34.

Domshlak, C. (2013). Fault tolerant planning: Complexity and compilation. In *ICAPS 2013 - Proceedings of the 23rd International Conference on Automated Planning and Scheduling*, pp. 64–72.

Draper, D., Hanks, S., & Weld, D. (1994). Probabilistic Planning with Information Gathering and Contingent Execution Representation : Actions & Contexts Actions. *Science*, *0*(0), 0.

Edelkamp, S., Lafuente, A. L., & Leue, S. (2001). Directed explicit model checking with HSF-SPIN. *Lecture Notes in Computer Science*, *2057*, 57–79.

Fikes, R. E., & Nilsson, N. J. (1971). Strips: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, *2*(3-4), 189–208.

Filippidis, I., Dimarogonas, D. V., & Kyriakopoulos, K. J. (2012). Decentralized multi-agent control from local LTL specifications. In *Proceedings of the IEEE Conference on Decision and Control*, pp. 6235–6240.

Guo, M., & Dimarogonas, D. V. (2015). Multi-agent plan reconfiguration under local LTL specifications. *International Journal of Robotics Research*, *34*(2), 218–235.

Hoffmann, J. (2003). The metric-FF planning system: Translating "ignoring delete lists" to numeric state variables. *Journal of Artificial Intelligence Research*, *20*, 291–341.

Hoffmann, J., & Brafman, R. I. (2006). Conformant planning via heuristic forward search: A new approach. *Artificial Intelligence*, *170*(6-7).

Karpas, E., Shleyfman, A., & Tennenholtz, M. (2017). Automated verification of social law robustness in strips. In *Proceedings International Conference on Automated Planning and Scheduling, ICAPS*, pp. 163–171.

Katz, M., Sohrabi, S., Samulowitz, H., & Sievers., S. (2018). Delfi: Online Planner Selection for Cost-Optimal Planning. *The 9th International Planning Competition, Description of Participant Planners of the Classical Track, IPC2018-booklet,*, -(-), 57–68.

Keller, R. M. (1976). Formal Verification of Parallel Programs. *Communications of the ACM*, *19*(7), 371–384.

Keren, S., Gal, A., & Karpas, E. (2015). Goal recognition design for non-optimal agents. In *Proceedings of the National Conference on Artificial Intelligence*, Vol. 5, pp. 3298–3304.

Keren, S., Gal, A., & Karpas, E. (2019). Goal recognition design in deterministic environments. *Journal of Artificial Intelligence Research*, *65*, 209–269.

Kovacs, D. L. (2012). A Multi-Agent Extension of PDDL3.1. Tech. rep..

Kress-Gazit, H., Lahijanian, M., & Raman, V. (2018). Synthesis for Robots: Guarantees and Feedback for Robot Behavior. *Annual Review of Control, Robotics, and Autonomous Systems*, -(-), 211–236.

Kupferman, O., & Vardi, M. Y. (2001). Model checking of safety properties. *Formal Methods in System Design*, *19*(3), 291–314.

Moarref, S., & Kress-Gazit, H. (2018). Reactive synthesis for robotic swarms. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, -(-), 71–87.

Muise, C., Felli, P., Miller, T., Pearce, A. R., & Sonenberg, L. (2016). Planning for a single agent in a multi-agent environment using FOND. In *IJCAI International Joint Conference on Artificial Intelligence*, Vol. 2016-Janua, pp. 3206–3212.

Nakhost, H., Hoffmann, J., & Müller, M. (2012). Resource-constrained planning: A Monte Carlo random walk approach. In *ICAPS 2012 - Proceedings of the 22nd International Conference on Automated Planning and Scheduling*, pp. 181–189.

Nir, R., & Karpas, E. (2019). Automated Verification of Social Laws for Continuous Time Multi-Robot Systems. *AAAI 2019*, *33*, 7683–7690.

Nir, R., Shleyfman, A., & Karpas, E. (2020). Automated Synthesis of Social Laws in STRIPS. *AAAI 2020*, *34*(06), 9941–9948.

Nir, R., Shleyfman, A., & Karpas, E. (2021). Learning-based Synthesis of Social Laws in STRIPS. *SoCS 2021*, -(-), –.

Nissim, R., Apsel, U., & Brafman, R. (2012). Tunneling and decomposition-based state reduction for optimal planning. In *Frontiers in Artificial Intelligence and Applications*, Vol. 242, pp. 624–629.

Patrizi, F., Lipoveztky, N., De Giacomo, G., & Geffner, H. (2011). Computing infinite plans for LTL goals using a classical planner. In *IJCAI International Joint Conference on Artificial Intelligence*, pp. 2003–2008.

Russell, S., & Norvig, P. (2016). *Artificial intelligence: a modern approach*.

Schillinger, P., Bürger, M., & Dimarogonas, D. V. (2018). Decomposition of Finite LTL Specifications for Efficient Multi-agent Planning. In *Springer Proceedings in Advanced Robotics*, Vol. 6, pp. 253–267. Springer Science and Business Media B.V.

Shoham, Y., & Tennenholtz, M. (1992). On the synthesis of useful social laws for artificial agent societies (preliminary report). *Proceedings Tenth National Conference on Artificial Intelligence*, -(-), 276–281.

Tuisov, A., & Karpas, E. (2020). Automated verification of social law robustness for reactive agents. In *24th European Conference on Artificial Intelligence - ECAI*, Vol. 325, pp. 2386–2393.